

SAPIENZA
UNIVERSITÀ DI ROMA

Integer Linear Programming

Volume 4 — MIP heuristics

Rounding, diving, feasibility pump, RINS, local branching, relax-and-fix and fix-and-optimize. Examples and exercises with complete solutions.

Fabio Furini

fabiofurini.github.io

Department of Computer, Control and Management Engineering — Sapienza University of Rome

Online version and downloadable material:
fabiofurini.github.io/integer-linear-programming/
github.com/fabiofurini/integer-linear-programming

Academic year 2026–2027

Contents

V MIP heuristics **2**

10 MIP heuristics **3**

10.1 MIP-based heuristics 4
10.2 The problems and their MIP models 5
10.3 Rounding 8
10.4 Diving 10
10.5 The feasibility pump 14
10.6 Improving an incumbent with a sub-MIP 19
10.7 Relax-and-fix and fix-and-optimize 26
10.8 Heuristics inside the solver 30
10.9 What MIP-based heuristics give 32
10.10`Python` implementation 33
10.11Typical mistakes 42
10.12In summary 43

Part V

MIP heuristics

Chapter 10

MIP heuristics

Prerequisites

- From this module, the chapters «LP relaxation and dual bounds» and «LP-based branch-and-bound algorithms»: the LP relaxation, the primal and dual bounds, the optimality gap, and the branching operations.
- From this module, the chapter «What a MIP solver does»: the incumbent solution, the sub-problems solved by a solver, and the heuristics inside it.
- From the module *Algorithms*: constructive heuristics (GR, FF, NN, LPT), local search and metaheuristics, whose solutions are the starting points of this chapter.
- From the module *Linear Programming*: the dual simplex algorithm, which re-optimizes an LP after a change of a bound.

Objectives

At the end of this chapter you will be able to:

- explain how a heuristic algorithm uses the LP relaxation and a MIP solver;
- execute by hand rounding, diving, the feasibility pump, RINS, local branching and proximity search on small instances;
- execute relax-and-fix and fix-and-optimize on a lot sizing instance;
- decide which method to use when a feasible solution is missing, or when an incumbent solution must be improved;
- compute the gap of a heuristic solution when a dual bound is available.

10.1 MIP-based heuristics

- The heuristic algorithms of the previous notes — constructive heuristics, local search and metaheuristics — work directly on a representation of the solutions (a binary vector, a vector *bin*, a tour, a vector *mach*). They never look at a mathematical model of the problem, and they never use a solver.

- In this last set of notes on heuristic algorithms the problem is given as a **mixed-integer linear program** (MIP), and the heuristic uses two tools that come with the model:
  - the **LP relaxation**, whose optimal solution tells *where to look*: it is the solution of a simpler problem, often close to the good solutions of the original one;
  - the **MIP solver**, which can explore *exactly* a set of solutions far too large to be enumerated, provided that the set is described by linear constraints.

  Heuristics that combine a mathematical programming model with heuristic ideas are also called **matheuristics**.

- The methods of these notes are built on five ideas, one or two per section.

idea	methods
round the relaxation	rounding; diving (fix one variable, re-solve, repeat)
chase feasibility	the feasibility pump
fix where relaxation and incumbent agree	RINS
bound the distance from the incumbent	local branching; proximity search
decompose along the time	relax-and-fix; fix-and-optimize

- All the methods are written for a MIP in the following general form.

A **MIP** with N variables $\boldsymbol{x} = (x_1, x_2, \dots, x_N)$ reads

$$\min \quad f(\boldsymbol{x}) \tag{10.1a}$$

$$\text{subject to} \quad A\,\boldsymbol{x} \le b, \tag{10.1b}$$

$$l_j \le x_j \le u_j, \qquad \forall j \in \{1, 2, \dots, N\}, \tag{10.1c}$$

$$x_j \in \mathbb{Z}, \qquad \forall j \in \mathscr{D}, \tag{10.1d}$$

where f is a linear function and $\mathscr{D} \subseteq \{1, 2, \dots, N\}$ is the set of the indices of the **integer variables**. The feasible region of the **LP relaxation** is

$$\mathcal{L} = \left\{\boldsymbol{x} \in \mathbb{R}^N : A\,\boldsymbol{x} \le b,\ l \le \boldsymbol{x} \le u\right\},$$

and $z(\text{LP}) = \min\{f(\boldsymbol{x}) : \boldsymbol{x} \in \mathcal{L}\}$ is the value of the LP relaxation on the instance I.

- A maximization problem, such as the KP, is handled in the same way, exchanging min and max, $\leq$ and $\geq$ in the comparisons of objective values.

- Throughout the notes, $\tilde{\boldsymbol{x}}$ denotes an optimal solution of an LP relaxation, and $\bar{\boldsymbol{x}}$ a point whose integer components $\bar{x}_j$, $j \in \mathscr{D}$, are integer. When $\bar{\boldsymbol{x}}$ is a feasible solution, the best one known, it is called the **incumbent**, and $f(\bar{\boldsymbol{x}})$ is its value. The **rounding** of a real number v to the nearest integer is

$$[\,v\,] = \left\lfloor v + \frac{1}{2} \right\rfloor,$$

  so that a value $\frac{1}{2}$ is rounded up.

- A point $\bar{\boldsymbol{x}}$ with integer components on $\mathscr{D}$ is **completed** by fixing the integer variables to their values $\bar{x}_j$ and solving the LP on the continuous variables: either the LP has an optimal solution, and $\bar{\boldsymbol{x}}$ is turned into a feasible solution of the MIP, or the LP is infeasible, and so is $\bar{\boldsymbol{x}}$. When all the variables are integer, the completion simply checks the constraints.

- The examples of the notes are solved with the MIP solver Gurobi, used from Python through `gurobipy`, as in the chapter on Gurobi of the MIP modelling course. When an LP has several optimal solutions, the text reports the one returned by the solver; all the codes are in the last section.

10.2 The problems and their MIP models

- The heuristics are illustrated on the four problems of the previous notes, whose definitions and assumptions are given in the notes on combinatorial optimization problems, and on their reference instances, recalled in the following table with the values computed in the previous notes on heuristic algorithms.

problem	reference instance	values
KP	$p = (10, 7, 6, 4)$, $w = (5, 4, 3, 3)$, $c = 9$	GR: $\{1, 3\}$, profit 16; $z(I) = 17$, $\{1, 2\}$
BPP	$w = (4, 4, 5, 3, 2, 3)$, $c = 7$	FF: 4 bins; $z(I) = 3$
TSP	5 cities, as in the notes on definitions	NN from city 1: 25; $z(I) = 18$
$P \,\|\| C_{\max}$	$t = (5, 5, 4, 4, 3, 3, 3)$, $m = 3$	LPT: $J_1 = \{1, 5, 7\}$, $J_2 = \{2, 6\}$, $J_3 = \{3, 4\}$, 11; $z(I) = 9$

- The MIP models below are the classical ones, studied in the notes on ILP formulations of classical problems and in the MIP modelling course; we recall them in the form used by the codes of these notes.

10.2.1 The knapsack problem

With $x_j = 1$ if and only if item j is selected:

$$\max \quad \sum_{j \in \mathscr{I}} p_j \, x_j \tag{10.2a}$$

$$\text{subject to} \quad \sum_{j \in \mathscr{I}} w_j \, x_j \leq c, \tag{10.2b}$$

$$x_j \in \{0,1\}, \qquad \forall j \in \mathscr{I}. \tag{10.2c}$$

10.2.2 The bin packing problem

With a set $\mathscr{B} = \{1, 2, \ldots, n\}$ of available bins, $x_{jb} = 1$ if and only if item j is packed into bin b, and $y_b = 1$ if and only if bin b is used:

$$\min \quad \sum_{b \in \mathscr{B}} y_b \tag{10.3a}$$

$$\text{subject to} \quad \sum_{b \in \mathscr{B}} x_{jb} = 1, \qquad \forall j \in \mathscr{I}, \tag{10.3b}$$

$$\sum_{j \in \mathscr{I}} w_j \, x_{jb} \leq c \, y_b, \qquad \forall b \in \mathscr{B}, \tag{10.3c}$$

$$x_{jb} \in \{0,1\}, \qquad \forall j \in \mathscr{I}, \ \forall b \in \mathscr{B}, \tag{10.3d}$$

$$y_b \in \{0,1\}, \qquad \forall b \in \mathscr{B}. \tag{10.3e}$$

- The content of bin b is the packing pattern $P_b = \{j \in \mathscr{I} : x_{jb} = 1\}$. Any upper bound on the number of bins can be used as $|\mathscr{B}|$; we use n, which is always enough.

10.2.3 The travelling salesman problem

Let $E = \big\{\{i,j\} : i,j \in \mathscr{V}, \ i < j\big\}$ be the set of the edges, $\delta(i)$ the set of the edges incident to city i, and $E(S)$ the set of the edges with both endpoints in $S \subseteq \mathscr{V}$. With $x_{ij} = 1$ if and only if the tour uses the edge $\{i,j\}$:

$$\min \quad \sum_{\{i,j\} \in E} d_{ij} \, x_{ij} \tag{10.4a}$$

$$\text{subject to} \quad \sum_{\{i,j\} \in \delta(i)} x_{ij} = 2, \qquad \forall i \in \mathscr{V}, \tag{10.4b}$$

$$\sum_{\{i,j\} \in E(S)} x_{ij} \leq |S| - 1, \qquad \forall S \subset \mathscr{V}, \ 2 \leq |S| \leq n-2, \tag{10.4c}$$

$$x_{ij} \in \{0,1\}, \qquad \forall \{i,j\} \in E. \tag{10.4d}$$

- The **degree constraints** (10.4b) say that every city is the endpoint of two edges of the tour; the **subtour elimination constraints** (10.4c) forbid closed subtours on a proper subset S of the cities. They are exponentially many, and on large instances they are generated only when violated.

10.2.4 The scheduling problem $P \,||\, C_{\max}$

With $x_{jh} = 1$ if and only if job j is assigned to machine h, and $C_{\max}$ the makespan:

$$\min \quad C_{\max} \tag{10.5a}$$

$$\text{subject to} \quad \sum_{h=1}^{m} x_{jh} = 1, \qquad \forall j \in \mathscr{J}, \tag{10.5b}$$

$$\sum_{j \in \mathscr{J}} t_j \, x_{jh} \le C_{\max}, \qquad \forall h \in \{1, 2, \ldots, m\}, \tag{10.5c}$$

$$x_{jh} \in \{0, 1\}, \qquad \forall j \in \mathscr{J}, \ \forall h \in \{1, 2, \ldots, m\}, \tag{10.5d}$$

$$C_{\max} \ge 0. \tag{10.5e}$$

- $C_{\max}$ is the only continuous variable of the four models: given the assignment, the completion sets it to the largest load $L_h = \sum_{j \in \mathscr{J}} t_j \, x_{jh}$.

10.2.5 The LP relaxations on the reference instances

- The following table reports the size of the four models on the reference instances, the value $z(\text{LP})$ of their LP relaxations, computed with Gurobi, and the optimal value $z(I)$.

problem	variables	constraints	$z(\text{LP})$	$z(I)$	LP solution integer?
KP	4	1	17.75	17	no
BPP	42	12	3	3	no
TSP	10	25	18	18	yes
$P \,||\, C_{\max}$	22	10	9	9	no

- On all four instances the LP value, rounded in the direction of the bound (down for the KP, up for the three minimization problems), is already the optimal value. The relaxations are *strong* on these instances: the difficulty is to find a solution that attains the bound, not to prove it.

- This is the typical situation in which MIP-based heuristics are useful.

- On the TSP reference instance the LP solution is itself the optimal tour $(1, 3, 5, 2, 4)$: every method of these notes finds it at once. With 5 cities, moreover, every integer solution of the degree constraints is a tour, since a subtour needs at least 3 cities and the remaining 2 cities cannot form another one. The TSP is therefore not used in the examples of the following sections.

Example 10.1: The LP solutions of the reference instances

KP. The LP solution is $\tilde{\boldsymbol{x}} = \left(1, \frac{1}{4}, 1, 0\right)$: the items are taken by non-increasing efficiency, and the split item 2 is taken for the fraction $\frac{9-8}{4} = \frac{1}{4}$ of its weight, so $z(\text{LP}) = 10 + \frac{7}{4} + 6 = 17.75$.

$P \,||\, C_{\max}$**.** The LP solution returned by the solver is (zeros omitted)

	$j=1$	$j=2$	$j=3$	$j=4$	$j=5$	$j=6$	$j=7$
$\tilde{x}_{j1}$		$\frac{3}{5}$				1	1
$\tilde{x}_{j2}$			$\frac{1}{2}$	1	1		
$\tilde{x}_{j3}$	1	$\frac{2}{5}$	$\frac{1}{2}$				

with loads $L_1 = 3 + 5 \cdot \frac{3}{5} + 3 = 9$, $L_2 = 4 \cdot \frac{1}{2} + 4 + 3 = 9$, $L_3 = 5 + 5 \cdot \frac{2}{5} + 4 \cdot \frac{1}{2} = 9$ and $\tilde{C}_{\max} = 9 = \frac{27}{3}$. The LP has infinitely many optimal solutions: every fractional assignment with three loads equal to 9 is optimal.

BPP. The LP solution returned by the solver puts item 1 into bin 6, item 2 into bin 4, item 3 for $\frac{4}{5}$ into bin 3 and for $\frac{1}{5}$ into bin 4, item 4 into bin 6, item 5 into bin 1, item 6 into bin 3, and opens the bins by the fractions

$$y^* = \left(\frac{2}{7}, 0, 1, \frac{5}{7}, 0, 1\right),$$

just enough to contain their loads: 2, 0, 7, 5, 0, 7. Its value is $\frac{21}{7} = 3$.

10.3 Rounding

Rounding is the simplest MIP-based heuristic. It solves the LP relaxation, rounds every integer variable of the LP solution to the nearest integer, and completes the continuous variables. It returns a feasible solution, or it fails if the rounded point violates a constraint.

- The pseudo-code of rounding reads as follows:

Algorithm

Algorithm: Rounding

Input: MIP (10.1) with integer variables $\mathscr{D}$

Output: Feasible solution $\bar{\boldsymbol{x}}$, or FAIL

```
Function Rounding(MIP)
    x̃ := optimal solution of the LP relaxation          // min{f(x) : x ∈ ℒ}
    for j ∈ 𝒟 do
        x̄_j ← [x̃_j]                                    // round to the nearest integer
    end
    fix x_j := x̄_j for all j ∈ 𝒟 and solve the LP on the continuous variables
    if the LP is feasible then
        return x̄ completed with the continuous part of the LP solution
    else
        return FAIL                                    // the rounded point violates a constraint
    end
```

- **Why it can work.** When the formulation is strong, the LP solution is close to an optimal integer solution, and many of its integer variables are already integer: rounding changes only a few of them, by less than $\frac{1}{2}$ each.

- **Why it can fail.** Rounding treats the variables one at a time, while the constraints link them. A constraint with all non-negative coefficients and right-hand side $b_i \geq 0$ stays satisfied if the variables are rounded *down*, and one of the form $\geq$ if they are rounded *up*.

- An **equality** constraint, such as the assignment constraints of the BPP and of $P\,||\,C_{\max}$, is almost always violated by independent roundings.

Observation 10.1

Let all the coefficients of A be non-negative, $b \geq 0$ and $l = 0$. If $\tilde{\boldsymbol{x}} \in \mathcal{L}$, then the point $\lfloor \tilde{\boldsymbol{x}} \rfloor$, obtained by rounding down every integer variable, belongs to $\mathcal{L}$.

Proof. For every j, $0 \leq \lfloor \tilde{x}_j \rfloor \leq \tilde{x}_j \leq u_j$, so the bounds hold. For every row i, since $a_{ij} \geq 0$, $\sum_j a_{ij} \lfloor \tilde{x}_j \rfloor \leq \sum_j a_{ij} \tilde{x}_j \leq b_i$. □

- For the KP, rounding the LP solution down gives the items that precede the split item: always feasible, and in general not optimal.

Example 10.2: Rounding on the reference instances

KP. The LP solution $\tilde{\boldsymbol{x}} = \left(1, \frac{1}{4}, 1, 0\right)$ is rounded to $\bar{\boldsymbol{x}} = (1, 0, 1, 0)$: weight $8 \leq 9$, profit 16. It is the solution of *GR* of the notes on constructive heuristics. Rounding $\tilde{x}_2 = \frac{1}{4}$ up instead would give $(1, 1, 1, 0)$, of weight $12 > 9$: infeasible.

$P\,||\,C_{\max}$. In the LP solution of Example 10.1, $\left[\frac{3}{5}\right] = 1$ and $\left[\frac{2}{5}\right] = 0$ send job 2 to machine 1, but $\left[\frac{1}{2}\right] = 1$ assigns job 3 to *both* machines 2 and 3:

$$\bar{x}_{31} + \bar{x}_{32} + \bar{x}_{33} = 0 + 1 + 1 = 2 \neq 1.$$

The assignment constraint (10.5b) of job 3 is violated: **rounding fails**.

BPP. Rounding the LP solution of Example 10.1 gives $\bar{\boldsymbol{y}} = (0, 0, 1, 1, 0, 1)$ and puts item 3 into bin 3. Two capacity constraints (10.3c) are violated: bin 1 contains item 5 but $\bar{y}_1 = 0$ ($2 \leq 0$ is false), and bin 3 contains items 3 and 6 ($5 + 3 = 8 > 7$). **Rounding fails**.

- The three outcomes summarize the method: on the KP, with a single packing constraint, rounding is safe (Observation 10.1); on the problems with assignment constraints it fails, because the LP spreads a job or an item over several machines or bins, and rounding cannot decide where it goes.

10.4 Diving

- Rounding takes all its decisions at once, on the basis of a single LP solution. **Diving** takes them one at a time, and after each decision it asks the LP again: the LP solution is re-computed with the decisions taken so far, and it adapts to them.

Diving repeatedly solves the LP relaxation, selects one integer variable with a fractional value, fixes it to an integer value, and solves the LP again. It stops with a feasible solution when the LP solution is integer on all the integer variables, and it fails when the LP becomes infeasible. A *rule* decides which variable is fixed and to which value.

- The pseudo-code of diving reads as follows:

Algorithm

Algorithm: Diving

Input: MIP (10.1); a rule; for guided diving, an incumbent $\bar{\boldsymbol{x}}$
Output: Feasible solution, or FAIL

Function *Diving(MIP, rule)*
  while *true* **do**
    solve the LP relaxation with the current bounds l, u
    if *the LP is infeasible* **then**
      return *FAIL* `// the dive ends in an infeasible LP`
    end
    $\tilde{\boldsymbol{x}} :=$ optimal solution of the LP
    $F := \{j \in \mathscr{D} : \tilde{x}_j \notin \mathbb{Z}\}$ `// fractional integer variables`
    if $F = \emptyset$ **then**
      return $\tilde{\boldsymbol{x}}$ `// integer on` $\mathscr{D}$`: feasible`
    end
    select $j \in F$ and a value $v \in \mathbb{Z}$ according to the rule
    $l_j \leftarrow v$; $\quad u_j \leftarrow v$ `// fix` $x_j := v$
  end

- **Fractional diving.** Select the variable that is *least* fractional, i.e., the $j \in F$ that minimizes $\left|\tilde{x}_j - [\,\tilde{x}_j\,]\right|$, and fix it to $v = [\,\tilde{x}_j\,]$. The LP is almost sure about this variable, so the fixing changes the LP solution as little as possible. In the codes of these notes, ties are broken in favour of the variable that comes first in the order of the variables ($x_{11}, x_{12}, \dots$ for the assignment models, then $C_{\max}$ or $y_1, y_2, \dots$).

- **Guided diving** [3]. With an incumbent $\bar{\boldsymbol{x}}$, select the $j \in F$ that minimizes $|\tilde{x}_j - \bar{x}_j|$, and fix it to $v = \bar{x}_j$. The incumbent guides the dive toward a region of good solutions, and the LP decides in which order its values are adopted.

- Many other rules exist: *coefficient diving* fixes the variable that appears in the fewest constraints that can be violated by rounding it; *pseudo-cost diving* uses the history of the branch-and-bound tree; *vector-length diving* is tailored to set covering constraints. Modern solvers run several of them.

- **Diving and branch-and-bound.** Fixing $x_j := [\,\tilde{x}_j\,]$ is a branching decision. A dive is a single path from the root of the branch-and-bound tree to a leaf, always taking one of the two children, without ever coming back: depth-first search without backtracking.

- **Cost.** Every step solves an LP that differs from the previous one by a single bound, so the dual simplex method re-optimizes it starting from the previous basis in a few iterations. A dive with $|\mathscr{D}|$ integer variables needs at most $|\mathscr{D}|$ LPs, usually far fewer, since an LP solution has many integer components already.

- **Failure.** The LP becomes infeasible when the fixings taken so far are incompatible. Implementations then undo the last fixing and try the other value (a single backtrack), or give up.

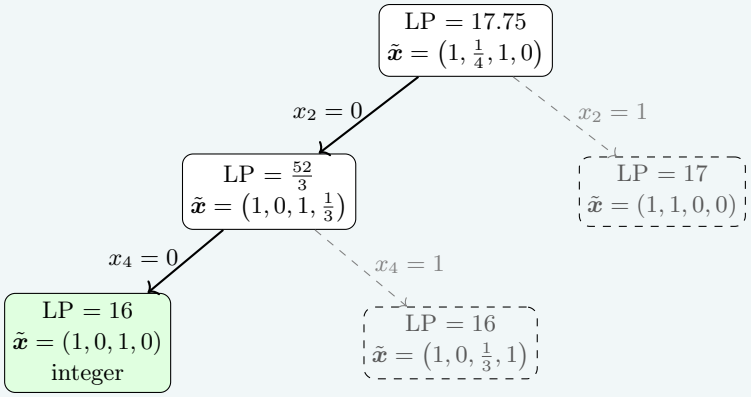
Example 10.3: Fractional diving on the KP

Step 1. The LP solution is $\tilde{\boldsymbol{x}} = \left(1, \frac{1}{4}, 1, 0\right)$, of value 17.75. The only fractional variable is x_2, rounded to $\left[\frac{1}{4}\right] = 0$: fix $x_2 := 0$.

Step 2. Without item 2, the LP takes items 1 and 3 (weight 8) and the fraction $\frac{1}{3}$ of item 4: $\tilde{\boldsymbol{x}} = \left(1, 0, 1, \frac{1}{3}\right)$, of value $10 + 6 + \frac{4}{3} = \frac{52}{3} \approx 17.33$. Fix $x_4 := \left[\frac{1}{3}\right] = 0$.

Step 3. The LP solution $\tilde{\boldsymbol{x}} = (1, 0, 1, 0)$, of value 16, is integer: the dive returns $\{1, 3\}$, of profit 16.

The dive is the path drawn with solid lines in the branch-and-bound tree below; the dashed nodes are the branches it never visits, with the value of their LP relaxations.



The branch $x_2 = 1$, abandoned at the first step, contains the optimal solution $\{1, 2\}$ of profit 17, which is an optimal solution of its LP relaxation. The first decision of the dive, $\left[\frac{1}{4}\right] = 0$, was the wrong one.

Example 10.4: Fractional diving on $P \,||\, C_{\max}$

The dive starts from the LP solution of Example 10.1. The table reports, for each step, the value of the LP, its fractional variables and the fixing; a fractionality $|\tilde{x}_j - [\tilde{x}_j]|$ equal for two variables is broken by the order $x_{11}, x_{12}, x_{13}, x_{21}, \ldots$

step	LP value	fractional variables	fixing
1	9	$x_{21} = \frac{3}{5}$, $x_{23} = \frac{2}{5}$, $x_{32} = \frac{1}{2}$, $x_{33} = \frac{1}{2}$	$x_{21} := 1$
2	9	$x_{11} = \frac{1}{5}$, $x_{13} = \frac{4}{5}$, $x_{32} = \frac{3}{4}$, $x_{33} = \frac{1}{4}$	$x_{11} := 0$
3	9	$x_{32} = \frac{3}{4}$, $x_{33} = \frac{1}{4}$, $x_{41} = \frac{1}{4}$, $x_{43} = \frac{3}{4}$	$x_{32} := 1$
4	9	$x_{41} = \frac{1}{4}$, $x_{43} = \frac{3}{4}$, $x_{52} = \frac{2}{3}$, $x_{53} = \frac{1}{3}$	$x_{41} := 0$
5	9	$x_{61} = \frac{1}{3}$, $x_{62} = \frac{2}{3}$	$x_{61} := 0$
6	9	$x_{12} = \frac{3}{5}$, $x_{13} = \frac{2}{5}$, $x_{71} = \frac{1}{3}$, $x_{72} = \frac{2}{3}$	$x_{71} := 0$
7	9.5	$x_{42} = \frac{1}{8}$, $x_{43} = \frac{7}{8}$	$x_{42} := 0$
8	9.5	$x_{62} = \frac{1}{6}$, $x_{63} = \frac{5}{6}$	$x_{62} := 0$
9	9.5	$x_{12} = \frac{1}{2}$, $x_{13} = \frac{1}{2}$	$x_{12} := 1$
10	9.5	$x_{72} = \frac{1}{6}$, $x_{73} = \frac{5}{6}$	$x_{72} := 0$
11	10	none	—

The last LP solution is integer: $J_1 = \{2, 5\}$, $J_2 = \{1, 3\}$, $J_3 = \{4, 6, 7\}$, with loads 8, 9, 10 and makespan 10. The dive improves on rounding, which failed, and on LPT (11), but it does not reach the optimum 9.

- For six steps the LP value stays at 9: the LP of $P\,||\,C_{\max}$ has a huge set of optimal solutions, and fixing one variable just moves the LP to another of them. The LP value tells nothing about the quality of the decisions until the load of a machine is forced above 9, at step 7, and then it is too late to undo the decisions that caused it. Min-max objectives are as flat for the LP as they are for local search.

Example 10.5: Guided diving on $P\,||\,C_{\max}$

The incumbent is the solution of LPT, $J_1 = \{1, 5, 7\}$, $J_2 = \{2, 6\}$, $J_3 = \{3, 4\}$, of makespan 11, i.e., $\bar{x}_{11} = \bar{x}_{51} = \bar{x}_{71} = \bar{x}_{22} = \bar{x}_{62} = \bar{x}_{33} = \bar{x}_{43} = 1$ and all the other $\bar{x}_{jh} = 0$.

Step 1. In the LP solution of Example 10.1 the fractional variables and their distances from the incumbent are

$$|\tilde{x}_{21} - \bar{x}_{21}| = \tfrac{3}{5}, \quad |\tilde{x}_{23} - \bar{x}_{23}| = \tfrac{2}{5}, \quad |\tilde{x}_{32} - \bar{x}_{32}| = \tfrac{1}{2}, \quad |\tilde{x}_{33} - \bar{x}_{33}| = \tfrac{1}{2}.$$

The closest is x_{23}: fix $x_{23} := \bar{x}_{23} = 0$.

Step 2. The new LP solution, of value 9, has the fractional variables $x_{12} = \frac{2}{5}$, $x_{13} = \frac{3}{5}$, $x_{31} = \frac{1}{2}$, $x_{32} = \frac{1}{2}$, at distances $\frac{2}{5}$, $\frac{3}{5}$, $\frac{1}{2}$, $\frac{1}{2}$ from the incumbent. Fix $x_{12} := \bar{x}_{12} = 0$.

Step 3. The LP solution is integer, of value 9: $J_1 = \{5, 6, 7\}$, $J_2 = \{2, 4\}$, $J_3 = \{1, 3\}$, all loads equal to 9. Guided diving finds an **optimal** solution after two LPs.

- Guided by a solution of makespan 11, the dive ends at a solution of makespan 9: the incumbent only decides the *values* of the two fixed variables, both equal to 0, while the LP rebuilds the rest of the assignment.

- On the BPP reference instance, fractional diving takes 8 fixings, all with LP value 3; the ninth, $x_{12} := 1$, puts item 1 into bin 2, which already contains item 2 by an earlier fixing, and $4+4>7$ makes the LP infeasible: **the dive fails**. The fixings $x_{jb} = 1$ commit items to bins one at a time, and the LP of the BPP is so weak (3 at every step) that it gives no warning before the dead end.

10.5 The feasibility pump

- Rounding and diving fail on the BPP reference instance, and on many real MIPs they fail most of the time: for a general MIP, even *deciding whether a feasible solution exists* is NP-complete. The **feasibility pump** of Fischetti, Glover and Lodi [4] gives up the objective function and concentrates on a single goal: to find a feasible solution.

- The idea is to work with two points at the same time. A point of $\mathcal{L}$ satisfies all the linear constraints, but may be fractional; an integer point is integer, but may violate the constraints. A feasible solution is a point that is both. The pump makes the two points chase each other until they coincide, and to do so it needs to measure how far apart they are.

Definition 10.2: Distance from an integer point

Let $\bar{\boldsymbol{x}}$ be a point with $\bar{x}_j \in \mathbb{Z}$ for all $j \in \mathscr{D}$. The **distance** of a point $\boldsymbol{x} \in \mathbb{R}^N$ from $\bar{\boldsymbol{x}}$ is

$$\Delta(\boldsymbol{x}, \bar{\boldsymbol{x}}) = \sum_{j \in \mathscr{D}} |x_j - \bar{x}_j|,$$

the L_1 distance computed on the integer variables only.

- When all the integer variables are **binary**, $|x_j - \bar{x}_j|$ equals x_j if $\bar{x}_j = 0$ and $1 - x_j$ if $\bar{x}_j = 1$, for every $x_j \in [0,1]$. The distance is then a *linear* function of $\boldsymbol{x}$:

$$\Delta(\boldsymbol{x}, \bar{\boldsymbol{x}}) = \sum_{j \in \mathscr{D}:\ \bar{x}_j = 0} x_j + \sum_{j \in \mathscr{D}:\ \bar{x}_j = 1} (1 - x_j) \qquad \text{for binary variables}$$

- For a general integer variable the same holds if $\bar{x}_j = l_j$ (the term is $x_j - l_j$) or $\bar{x}_j = u_j$ (the term is $u_j - x_j$). Otherwise $|x_j - \bar{x}_j|$ is replaced by an auxiliary variable d_j with the two constraints $d_j \geq x_j - \bar{x}_j$ and $d_j \geq \bar{x}_j - x_j$; at the minimum, $d_j = |x_j - \bar{x}_j|$. In all cases, the point of $\mathcal{L}$ closest to $\bar{\boldsymbol{x}}$ is computed by solving **one LP**.

The *feasibility pump* builds two sequences of points: points $\tilde{\boldsymbol{x}}^0, \tilde{\boldsymbol{x}}^1, \tilde{\boldsymbol{x}}^2, \ldots$ of $\mathcal{L}$, which satisfy the linear constraints, and integer points $\bar{\boldsymbol{x}}^0, \bar{\boldsymbol{x}}^1, \bar{\boldsymbol{x}}^2, \ldots$, which satisfy the integrality constraints. It starts from the LP solution $\tilde{\boldsymbol{x}}^0$; each integer point is the *rounding* of the last point of $\mathcal{L}$, and each point of $\mathcal{L}$ is the *projection* of the last integer point, i.e., the point of $\mathcal{L}$ at minimum distance from it. It stops when the distance is 0: the two points coincide and are a feasible solution. When the rounding returns an integer point already visited, the

pump *perturbs* it.

- The pseudo-code of the feasibility pump reads as follows:

Algorithm

Algorithm: Feasibility pump

Input: MIP (10.1); maximum number of iterations $maxIter$; seed of the random numbers

Output: Feasible solution, or FAIL

Function *FeasibilityPump(MIP, maxIter, seed)*
  $\tilde{\boldsymbol{x}}$:= optimal solution of the LP relaxation
  if $\tilde{x}_j \in \mathbb{Z}$ *for all* $j \in \mathscr{D}$ **then**
    return $\tilde{\boldsymbol{x}}$
  end
  $\bar{\boldsymbol{x}} \leftarrow [\tilde{\boldsymbol{x}}]$ `// round the integer components`
  for $it := 1$ **to** $maxIter$ **do**
    $\tilde{\boldsymbol{x}} \leftarrow \arg\min\{\Delta(\boldsymbol{x}, \bar{\boldsymbol{x}}) : \boldsymbol{x} \in \mathcal{L}\}$ `// projection: one LP`
    if $\Delta(\tilde{\boldsymbol{x}}, \bar{\boldsymbol{x}}) = 0$ **then**
      return $\tilde{\boldsymbol{x}}$ `// x̃ is integer on 𝒟: feasible`
    end
    $\hat{\boldsymbol{x}} \leftarrow [\tilde{\boldsymbol{x}}]$
    if $\hat{x}_j = \bar{x}_j$ *for all* $j \in \mathscr{D}$ **then** `// short cycle`
      $j \leftarrow \arg\max_{k \in \mathscr{D}} |\tilde{x}_k - \bar{x}_k|$
      move $\hat{x}_j$ by one unit toward $\tilde{x}_j$ `// flip`
    end
    if $\hat{\boldsymbol{x}}$ *equals one of the last* 3 *points* $\bar{\boldsymbol{x}}$ **then** `// long cycle`
      for $j \in \mathscr{D}$ **do**
        $\rho_j \leftarrow$ uniform random number in $[-0.3, 0.7]$
        if $|\tilde{x}_j - \hat{x}_j| + \max\{\rho_j, 0\} > \frac{1}{2}$ **then**
          $\hat{x}_j \leftarrow 1 - \hat{x}_j$ `// restart`
        end
      end
    end
    $\bar{\boldsymbol{x}} \leftarrow \hat{\boldsymbol{x}}$
  end
  return *FAIL*

- **Why the pump works.** Each projection moves the point of $\mathcal{L}$ toward the last integer point, and each rounding moves the integer point to the nearest one. The distance between the two sequences tends to decrease, and the points are *pumped* toward a region where $\mathcal{L}$ contains integer points. Each iteration costs one LP and a rounding.

- **Short cycles.** If the rounding of the projection gives back the same integer point, the pump would repeat the same two points forever. It then changes the components of $\bar{\boldsymbol{x}}$ on which $\tilde{\boldsymbol{x}}$ is most uncertain, those with the largest $|\tilde{x}_j - \bar{x}_j|$.

- The original pump flips a random number of components between 10 and 30. On the small instances of these notes one flip is enough: the code flips the single component with the largest $|\tilde{x}_j - \bar{x}_j|$ (the first one, in case of ties).

- **Long cycles.** The pump can also cycle among several integer points. When the new integer point equals one of the last ones, the original pump makes a **restart**: a binary component is flipped when $|\tilde{x}_j - \hat{x}_j| + \max\{\rho_j, 0\} > \frac{1}{2}$, with ρ_j uniformly random in $[-0.3, 0.7]$, so that the components where $\tilde{x}_j$ is far from $\hat{x}_j$ are flipped with high probability.

- **Random numbers.** In all the notes on heuristic algorithms, random numbers are produced by the *minimal standard* generator of Park and Miller, with integer arithmetic on 64 bits:
$$seed \leftarrow (16807 \cdot seed) \bmod (2^{31} - 1), \qquad u = \frac{seed}{2^{31} - 1} \in (0, 1),$$
and in the pump $\rho_j = u - 0.3$. Fixing the initial seed makes every run reproducible.

- **The objective.** After the first LP, the original objective f is ignored: the pump finds a feasible solution, but not necessarily a good one. The *objective feasibility pump* of Achterberg and Berthold [1] minimizes a combination $(1 - \alpha)\,\Delta(\boldsymbol{x}, \bar{\boldsymbol{x}}) + \alpha\, f(\boldsymbol{x})$ with a weight α that decreases at every iteration: the solutions found are much better, at the price of a few more iterations.

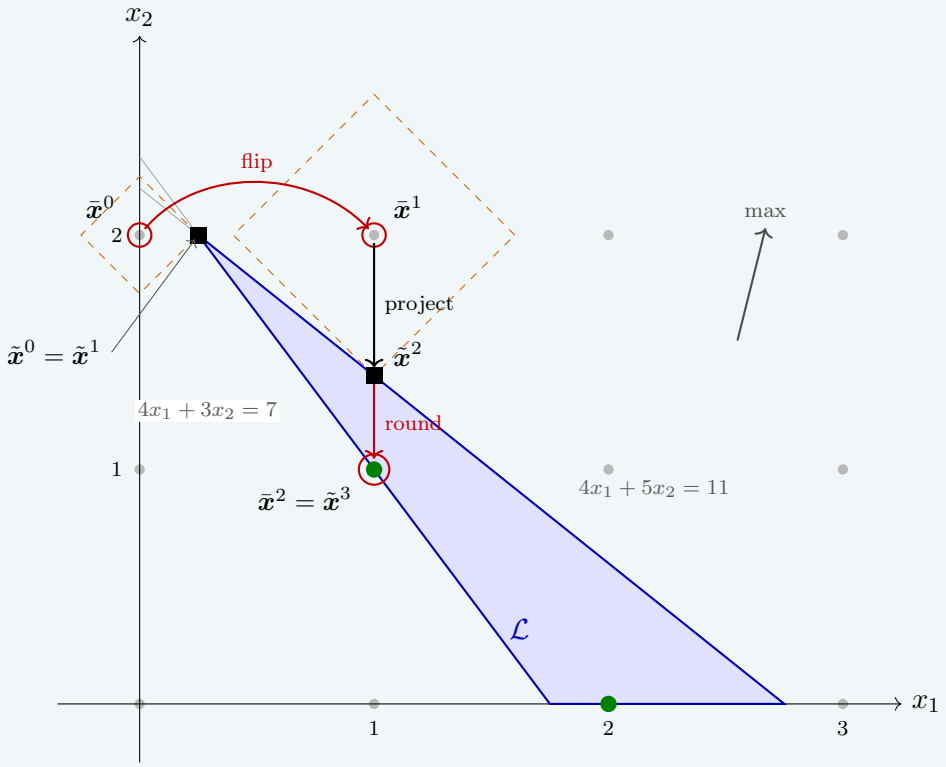
- **General integer variables.** The pump was designed for binary variables, where the flip changes $\bar{x}_j$ into $1 - \bar{x}_j$; for general integer variables it moves $\bar{x}_j$ by one unit toward $\tilde{x}_j$, and the distance uses the auxiliary variables d_j [2].

Example 10.6: The feasibility pump in two variables

Consider the integer program

$$\begin{aligned} \max \quad & x_1 + 4\,x_2 \\ \text{subject to} \quad & 4\,x_1 + 3\,x_2 \geq 7 \\ & 4\,x_1 + 5\,x_2 \leq 11 \\ & x_1 \in \{0, 1, \ldots, 5\} \\ & x_2 \in \{0, 1, \ldots, 5\} \end{aligned}$$

The region $\mathcal{L}$ is the triangle with vertices $\left(\frac{1}{4}, 2\right)$, $\left(\frac{7}{4}, 0\right)$, $\left(\frac{11}{4}, 0\right)$, which contains only two integer points, $(1, 1)$ and $(2, 0)$. The figure shows the two sequences of points: squares for the points of $\mathcal{L}$, circles for the integer points, and the diamonds $\{\boldsymbol{x} : \Delta(\boldsymbol{x}, \bar{\boldsymbol{x}}) \leq r\}$ of the two projections.



The iterations are the following.

k	integer point	projection $\tilde{\boldsymbol{x}}^k$	$\Delta(\tilde{\boldsymbol{x}}^k, \bar{\boldsymbol{x}}^{k-1})$	next integer point
0	—	LP optimum $\left(\frac{1}{4}, 2\right)$	—	rounding: $\bar{\boldsymbol{x}}^0 = (0, 2)$
1	$\bar{\boldsymbol{x}}^0 = (0, 2)$	$\left(\frac{1}{4}, 2\right)$	$\frac{1}{4}$	$\lfloor \tilde{\boldsymbol{x}}^1 \rfloor = \bar{\boldsymbol{x}}^0$: flip x_1, $\bar{\boldsymbol{x}}^1 = (1, 2)$
2	$\bar{\boldsymbol{x}}^1 = (1, 2)$	$\left(1, \frac{7}{5}\right)$	$\frac{3}{5}$	rounding: $\bar{\boldsymbol{x}}^2 = (1, 1)$
3	$\bar{\boldsymbol{x}}^2 = (1, 1)$	$(1, 1)$	0	stop: feasible

Iteration 0. The LP optimum is the vertex $\left(\frac{1}{4}, 2\right)$, of value $\frac{33}{4}$. Its rounding $(0, 2)$ violates the first constraint: $4 \cdot 0 + 3 \cdot 2 = 6 < 7$.

Iteration 1. With $\bar{\boldsymbol{x}}^0 = (0, 2)$, x_1 is at its lower bound and $\Delta(\boldsymbol{x}, \bar{\boldsymbol{x}}^0) = x_1 + |x_2 - 2|$. The L_1 distance from a point to a segment that is neither horizontal, vertical nor of slope ± 1 is minimized at an endpoint or where the segment crosses the horizontal or the vertical line through the point; so the candidates are the three vertices and these crossings. The horizontal line $x_2 = 2$ touches $\mathcal{L}$ only at the vertex $\left(\frac{1}{4}, 2\right)$, and the vertical line $x_1 = 0$ misses it; the closest point is $\tilde{\boldsymbol{x}}^1 = \left(\frac{1}{4}, 2\right)$, at distance $\frac{1}{4}$. Its rounding is $(0, 2) = \bar{\boldsymbol{x}}^0$ again: a short cycle. The component with the largest $|\tilde{x}^1_j - \bar{x}^0_j|$ is x_1 ($\frac{1}{4}$ against 0), which moves one unit toward $\frac{1}{4}$: $\bar{\boldsymbol{x}}^1 = (1, 2)$.

Iteration 2. The point $(1, 2)$ violates the second constraint ($14 > 11$). The vertical line $x_1 = 1$ crosses $\mathcal{L}$ between $(1, 1)$ and $\left(1, \frac{7}{5}\right)$, at distances 1 and $\frac{3}{5}$ from $(1, 2)$; the horizontal line $x_2 = 2$ gives $\left(\frac{1}{4}, 2\right)$, at distance $\frac{3}{4}$; the other two vertices are farther. Hence $\tilde{\boldsymbol{x}}^2 = \left(1, \frac{7}{5}\right)$, and its rounding $(1, 1)$ is a new point.

Iteration 3. The point $(1, 1)$ belongs to $\mathcal{L}$ ($4 + 3 = 7 \geq 7$ and $4 + 5 = 9 \leq 11$): its distance from $\mathcal{L}$ is 0, and the pump returns $(1, 1)$, of value 5. It is optimal, since the other integer point $(2, 0)$ has value 2.

Example 10.7: The feasibility pump on $P\,||\,C_{\max}$

Iteration 0. The LP solution of Example 10.1 is rounded as in Example 10.2: job 3 is assigned to machines 2 and 3, and $\bar{\boldsymbol{x}}^0$ is infeasible.

Iteration 1. The projection minimizes $\Delta(\boldsymbol{x}, \bar{\boldsymbol{x}}^0)$, which is linear since all the integer variables are binary. The solver returns the point of $\mathcal{L}$ that keeps all the assignments of $\bar{\boldsymbol{x}}^0$ and removes job 3 from machine 2, i.e., it moves x_{32} from 1 to 0: $\Delta = 1$ (removing job 3 from machine 3 would be equally close). The result is integer:

$$J_1 = \{2, 6, 7\}, \qquad J_2 = \{4, 5\}, \qquad J_3 = \{1, 3\},$$

with loads 11, 7, 9, and its rounding is the new integer point $\bar{\boldsymbol{x}}^1$.

Iteration 2. $\bar{\boldsymbol{x}}^1 \in \mathcal{L}$, the distance is 0, and the pump returns a solution of makespan 11, after two projections.

- The solution has the same makespan as LPT: the projection takes care of the assignment constraints and ignores the loads, since $C_{\max}$ is continuous and does not enter Δ. The pump answers the question "is there a feasible solution?", which for $P\,||\,C_{\max}$ was not a difficult question.

Example 10.8: The feasibility pump on the BPP

On the BPP reference instance, where rounding and diving fail, the pump is run with the seeds $1, 2, \ldots, 10$ of the random numbers. The table reports the number of projections, of flips and of restarts, the number of bins opened by the solution found ($\sum_b \bar{y}_b$, the value of the solution) and the number of non-empty bins.

seed	1	2	3	4	5	6	7	8	9	10
projections	15	12	9	9	6	6	19	5	10	20
flips	8	5	3	4	3	3	8	2	5	8
restarts	4	3	3	2	1	1	5	1	2	4
bins opened	4	4	4	5	5	4	3	4	4	5
non-empty bins	4	4	4	3	5	4	3	4	4	4

The pump finds a feasible solution with every seed, but always after some restarts: without them it would cycle. Only with seed 7 the solution is optimal: $\{3, 5\}$, $\{2, 4\}$, $\{1, 6\}$, of value 3. With seed 4 the pump opens 5 bins, two of which stay empty: the packing $\{3, 5\}$, $\{1, 6\}$, $\{2, 4\}$ is optimal, but the solution has value 5, because nothing in the distance function asks to close the empty bins.

- The pump is a method to find a *first* feasible solution, and it does that well. The quality of the solution is the job of the improvement methods of the next sections, which start from an incumbent.

10.6 Improving an incumbent with a sub-MIP

- The methods of the previous sections build a solution from the LP relaxation. The three methods of this section start from an **incumbent** $\bar{\boldsymbol{x}}$, computed by any heuristic — a constructive heuristic, a local search, the feasibility pump — and try to improve it.

- They all follow the same pattern: add to the MIP some constraints that restrict it to a set of solutions *related to* $\bar{\boldsymbol{x}}$, and solve the restricted problem, called the **sub-MIP**, with the MIP solver.

- The sub-MIP is a neighbourhood of $\bar{\boldsymbol{x}}$ in the sense of the notes on definitions, but a very large one, explored exactly by the solver rather than by enumeration. It is the idea of large neighbourhood search of the notes on metaheuristics, with the solver as the repair method.

- Only solutions *better* than the incumbent are of interest. With an improvement step $\theta > 0$, the sub-MIP contains the **cutoff constraint**

$$f(\boldsymbol{x}) \leq f(\bar{\boldsymbol{x}}) - \theta \quad \text{(minimization)}, \qquad f(\boldsymbol{x}) \geq f(\bar{\boldsymbol{x}}) + \theta \quad \text{(maximization)}.$$

- When f takes integer values on all the feasible solutions, as for the four problems of these notes, $\theta = 1$ excludes exactly the solutions that are not better than $\bar{\boldsymbol{x}}$. If the sub-MIP with the cutoff is infeasible, the neighbourhood contains no better solution: the solver has *proved* that $\bar{\boldsymbol{x}}$ is a local optimum for that neighbourhood.

- The methods differ in *which* constraints define the neighbourhood: RINS fixes variables (**hard fixing**), local branching and proximity search bound the number of variables that change (**soft fixing**).

10.6.1 RINS

RINS (Relaxation Induced Neighbourhood Search) [3] compares the incumbent $\bar{\boldsymbol{x}}$ with the solution $\tilde{\boldsymbol{x}}$ of the LP relaxation. The integer variables on which the two solutions *agree* are fixed to their common value; the sub-MIP on the remaining variables, with the cutoff constraint, is solved by the MIP solver.

- The pseudo-code of RINS reads as follows:

Algorithm

Algorithm: RINS

Input: MIP (10.1); incumbent $\bar{\boldsymbol{x}}$; improvement step θ
Output: A solution at least as good as $\bar{\boldsymbol{x}}$

Function *RINS(MIP, $\bar{\boldsymbol{x}}$, θ)*
  $\tilde{\boldsymbol{x}}$:= optimal solution of the LP relaxation
  for $j \in \mathscr{B}$ **do**
    if $\tilde{x}_j = \bar{x}_j$ **then**
      fix $x_j := \bar{x}_j$ `// LP and incumbent agree`
    end
  end
  add the cutoff constraint $f(\boldsymbol{x}) \le f(\bar{\boldsymbol{x}}) - \theta$
  solve the sub-MIP with the MIP solver (with a node limit)
  if *the sub-MIP has a feasible solution* $\boldsymbol{x}'$ **then**
    return $\boldsymbol{x}'$ `// better than` $\bar{\boldsymbol{x}}$
  else
    return $\bar{\boldsymbol{x}}$ `// no better solution in the neighbourhood`
  end

- **Why the agreement.** The LP solution is optimistic and possibly infeasible, the incumbent is feasible and possibly poor: two advisers with opposite defects. Where they agree, the value is probably right in good solutions too; where they disagree, there is something to decide, and the decision is left to the solver.

- **The size of the neighbourhood is not chosen.** It is the number of variables on which $\tilde{\boldsymbol{x}}$ and $\bar{\boldsymbol{x}}$ disagree: small if the incumbent is already close to the LP solution, large otherwise. For this reason the sub-MIP is solved with a limit on the number of nodes.

- **RINS inside branch-and-bound.** Danna, Rothberg and Le Pape apply RINS at the nodes of the branch-and-bound tree, with the LP solution of the node: deeper nodes have different LP solutions, hence different neighbourhoods of the same incumbent. Every modern MIP solver runs RINS in this way.

- **Hard fixing.** A fixed variable cannot change in the sub-MIP. If a better solution needs to change a variable on which $\tilde{\boldsymbol{x}}$ and $\bar{\boldsymbol{x}}$ agree, RINS cannot find it.

Example 10.9: RINS on the KP

The incumbent is the solution $\bar{\boldsymbol{x}} = (1,0,1,0)$ of GR, of profit 16, and the LP solution is $\tilde{\boldsymbol{x}} = \left(1, \frac{1}{4}, 1, 0\right)$. They agree on $x_1 = 1$, $x_3 = 1$, $x_4 = 0$, which are fixed. The sub-MIP

$$\max\ 16 + 7\,x_2 \quad \text{subject to} \quad 8 + 4\,x_2 \le 9, \quad 16 + 7\,x_2 \ge 17, \quad x_2 \in \{0,1\}$$

is infeasible: the capacity requires $x_2 = 0$, the cutoff $x_2 = 1$. RINS returns the incumbent.

The optimal solution $\{1, 2\}$ has $x_3 = 0$, but x_3 is fixed to 1 because the LP and the incumbent agree on it.

Example 10.10: RINS on $P \,||\, C_{\max}$

The incumbent is the solution of *LPT* ($J_1 = \{1, 5, 7\}$, $J_2 = \{2, 6\}$, $J_3 = \{3, 4\}$, makespan 11), and $\tilde{\boldsymbol{x}}$ is the LP solution of Example 10.1. The table compares, for each job, the machine of the incumbent with the LP solution.

	$j=1$	$j=2$	$j=3$	$j=4$	$j=5$	$j=6$	$j=7$
machine in $\bar{\boldsymbol{x}}$	1	2	3	3	1	2	1
machines in $\tilde{\boldsymbol{x}}$	3	1 ($\frac{3}{5}$), 3 ($\frac{2}{5}$)	2 ($\frac{1}{2}$), 3 ($\frac{1}{2}$)	2	2	1	1
fixed variables	$x_{12} = 0$	—	$x_{31} = 0$	$x_{41} = 0$	$x_{53} = 0$	$x_{63} = 0$	$x_{71} = 1$, $x_{72} = x_{73} = 0$

Only job 7 is on the same machine in both solutions; for the other jobs, the two solutions agree only on some machines that the job does *not* use. Eight of the 21 binary variables are fixed. The sub-MIP with the cutoff $C_{\max} \leq 10$ is solved by Gurobi, which returns

$$J_1 = \{5, 6, 7\}, \qquad J_2 = \{2, 3\}, \qquad J_3 = \{1, 4\},$$

with all loads equal to 9: **RINS finds an optimal solution**.

- On the KP the incumbent and the LP solution are too similar, and the neighbourhood of RINS has a single free variable; on $P \,||\, C_{\max}$ they are very different, and the neighbourhood is large enough to contain the optimum.

10.6.2 Local branching

- RINS decides *which* variables may change. Local branching [5] decides only *how many* of them may change, and leaves the choice of which ones to the solver. For a MIP whose integer variables are binary, the **local branching constraint** with radius $k \in \mathbb{Z}_{\geq 1}$ is

$$\Delta(\boldsymbol{x}, \bar{\boldsymbol{x}}) \;=\; \sum_{j \in \mathscr{D}:\ \bar{x}_j = 1} (1 - x_j) \;+\; \sum_{j \in \mathscr{D}:\ \bar{x}_j = 0} x_j \;\leq\; k$$

- $\Delta(\boldsymbol{x}, \bar{\boldsymbol{x}})$ is the distance of Definition 10.2 for binary variables: on binary points, it counts the variables whose value differs from the incumbent. The constraint is linear, so it can be added to the MIP like any other constraint.

Local branching solves, with the MIP solver, the sub-MIP made of the MIP, the cutoff constraint and the local branching constraint $\Delta(\boldsymbol{x}, \bar{\boldsymbol{x}}) \leq k$. If it finds a better solution, the new solution becomes the incumbent and the neighbourhood is centred on it. If it proves that the neighbourhood contains no better solution, the radius k is enlarged. In both cases the explored neighbourhood is excluded from the following sub-MIPs by the *reverse* constraint $\Delta(\boldsymbol{x}, \bar{\boldsymbol{x}}) \geq k + 1$.

Algorithm

Algorithm: Local branching

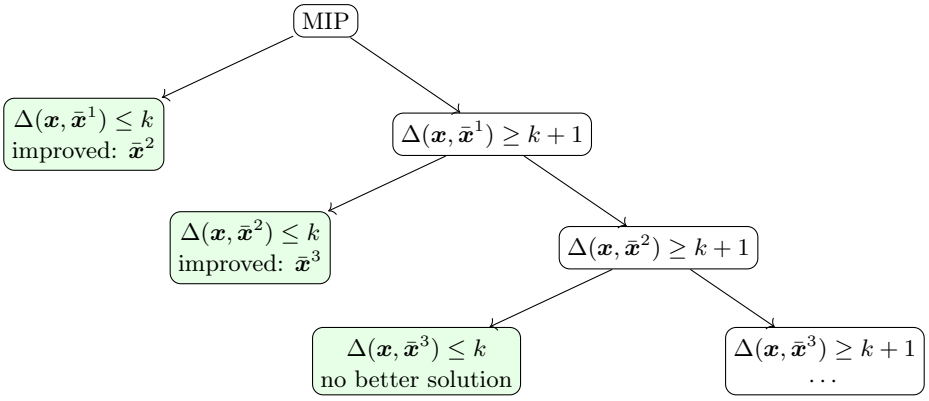
Input: MIP (10.1) with binary integer variables; incumbent $\bar{\boldsymbol{x}}$; radius k; maximum radius $k_{\max}$; step k_{step}; improvement step θ

Output: A solution at least as good as $\bar{\boldsymbol{x}}$

Function *LocalBranching(MIP,* $\bar{\boldsymbol{x}}$*,* k*,* $k_{\max}$*,* k_{step}*,* θ*)*
  $\mathscr{R} := \emptyset$ `// reverse constraints added so far`
  while $k \le k_{\max}$ **do**
    solve the MIP with the constraints $\mathscr{R}$, $\Delta(\boldsymbol{x}, \bar{\boldsymbol{x}}) \le k$ and $f(\boldsymbol{x}) \le f(\bar{\boldsymbol{x}}) - \theta$
    $\mathscr{R} \leftarrow \mathscr{R} \cup \{\Delta(\boldsymbol{x}, \bar{\boldsymbol{x}}) \ge k+1\}$ `// never explore it again`
    if *a feasible solution* $\boldsymbol{x}'$ *is found* **then**
      $\bar{\boldsymbol{x}} \leftarrow \boldsymbol{x}'$ `// re-centre the neighbourhood`
    else
      $k \leftarrow k + k_{step}$ `// no better solution within distance` k
    end
  end
  return $\bar{\boldsymbol{x}}$

- **Soft fixing.** With $|\mathscr{D}|$ binary variables, the neighbourhood $\Delta(\boldsymbol{x}, \bar{\boldsymbol{x}}) \le k$ contains $\sum_{i=0}^{k} \binom{|\mathscr{D}|}{i}$ binary points: for k small it is much smaller than the whole space, but it does not commit to any particular variable. Any k variables may change.

- **A branching scheme.** The name comes from the original use of the constraint as a *branching rule*: the left branch $\Delta(\boldsymbol{x}, \bar{\boldsymbol{x}}) \le k$ is solved first, and the right branch $\Delta(\boldsymbol{x}, \bar{\boldsymbol{x}}) \ge k+1$ contains the rest of the problem. If the left branch improves the incumbent, the right branch is split again around the new incumbent; the tree is a long chain of right branches:



- **The radius.** k must be small enough for the sub-MIP to be solved quickly, and large enough for the neighbourhood to contain better solutions. The original method uses small values compared with $|\mathscr{D}|$, a time limit on each sub-MIP, and *diversification* — a larger k, or a jump to a different solution — when no improvement is found.

- **Local branching and local search.** With $k = 1$ the neighbourhood is the flip neighbourhood of the KP in the notes on definitions; with $k = 2$ it contains the swap neighbourhood as well. For the assignment variables of the BPP and of $P\,||\,C_{\max}$ the distance has a simple meaning.

Observation 10.3

Let $\boldsymbol{x}$ and $\bar{\boldsymbol{x}}$ be two feasible assignments of $P\,||\,C_{\max}$. Then $\Delta(\boldsymbol{x}, \bar{\boldsymbol{x}}) = 2\,r$, where r is the number of jobs assigned to different machines in $\boldsymbol{x}$ and in $\bar{\boldsymbol{x}}$.

Proof. A job on the same machine in $\boldsymbol{x}$ and $\bar{\boldsymbol{x}}$ contributes 0 to Δ. A job on machine h in $\bar{\boldsymbol{x}}$ and on machine $h' \neq h$ in $\boldsymbol{x}$ contributes $1 - x_{jh} = 1$ and $x_{jh'} = 1$, and 0 for the other machines. □

- Hence, for $P\,||\,C_{\max}$, the neighbourhood $\Delta(\boldsymbol{x}, \bar{\boldsymbol{x}}) \leq 2$ is the **move** neighbourhood of the notes on local search (one job changes machine), and $\Delta(\boldsymbol{x}, \bar{\boldsymbol{x}}) \leq 4$ contains the **swap** neighbourhood (two jobs change machine). The same holds for the variables x_{jb} of the BPP.

Example 10.11: Local branching on the KP

Start from $\bar{\boldsymbol{x}}^1 = (1, 0, 1, 0)$, of profit 16, with $k = 1$, $k_{\max} = 3$, $k_{step} = 1$ and $\theta = 1$.

$k = 1$. The neighbours at distance 1 are the flips of $\bar{\boldsymbol{x}}^1$; with the cutoff $\sum_j p_j x_j \geq 17$ no one is feasible. Add $\Delta(\boldsymbol{x}, \bar{\boldsymbol{x}}^1) \geq 2$ and set $k = 2$.

$k = 2$. The sub-MIP with $2 \leq \Delta(\boldsymbol{x}, \bar{\boldsymbol{x}}^1) \leq 2$ finds the swap $(1, 1, 0, 0)$, of profit 17: the new incumbent $\bar{\boldsymbol{x}}^2$. Add $\Delta(\boldsymbol{x}, \bar{\boldsymbol{x}}^1) \geq 3$.

$k = 2$, around $\bar{\boldsymbol{x}}^2$. No solution of profit at least 18: add $\Delta(\boldsymbol{x}, \bar{\boldsymbol{x}}^2) \geq 3$ and set $k = 3$.
$k = 3$. No solution of profit at least 18 either; k becomes $4 > k_{\max}$ and the method stops with $\{1, 2\}$, of profit 17: the optimum.

Example 10.12: Local branching on $P\,||\,C_{\max}$

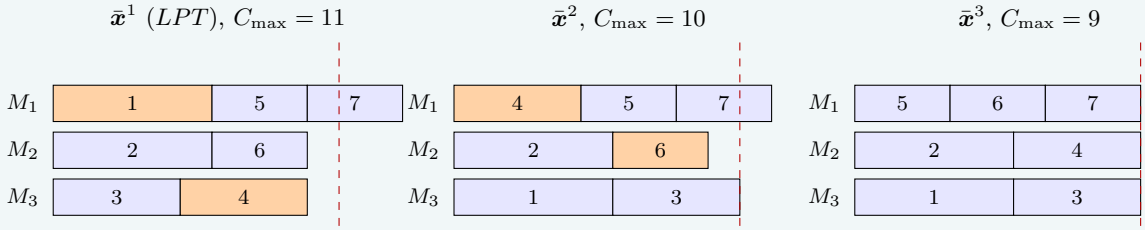
Start from the solution $\bar{\boldsymbol{x}}^1$ of LPT, of makespan 11, with $k = 2$, $k_{\max} = 6$, $k_{step} = 2$ and $\theta = 1$, so that the cutoff is $C_{\max} \leq 10$, then 9, and so on.

$k = 2$. By Observation 10.3 the neighbourhood is the move neighbourhood, and LPT is a local optimum for it (notes on local search): the sub-MIP is infeasible. Add $\Delta(\boldsymbol{x}, \bar{\boldsymbol{x}}^1) \geq 3$ and set $k = 4$.

$k = 4$. The best solution with two jobs reassigned has makespan 10; the solver returns the swap of jobs 1 and 4, $\bar{\boldsymbol{x}}^2$ below. Add $\Delta(\boldsymbol{x}, \bar{\boldsymbol{x}}^1) \geq 5$.

$k = 4$, around $\bar{\boldsymbol{x}}^2$. The solver returns the swap of jobs 4 and 6, of makespan 9: $\bar{\boldsymbol{x}}^3$. Add $\Delta(\boldsymbol{x}, \bar{\boldsymbol{x}}^2) \geq 5$.

$k = 4$ and $k = 6$, around $\bar{\boldsymbol{x}}^3$. No solution of makespan 8: k becomes $8 > k_{\max}$, and the method stops with makespan 9, the optimum.



The dashed line is the lower bound 9; the jobs swapped to obtain the next solution are coloured.

- In the language of the notes on local search, local branching performs a local search with a neighbourhood whose size is controlled by k, explored exactly by the solver, and it enlarges the neighbourhood when it gets stuck — exactly the idea of variable neighbourhood search, with the solver doing the exploration.

10.6.3 Proximity search

- Local branching keeps the objective and constrains the distance. **Proximity search** of Fischetti and Monaci [6] does the opposite: it keeps the cutoff as a constraint and *minimizes* the distance.

The **proximity sub-MIP** around the incumbent $\bar{\boldsymbol{x}}$ reads

$$\begin{aligned} \min \quad & \Delta(\boldsymbol{x}, \bar{\boldsymbol{x}}) \\ \text{subject to} \quad & A\,\boldsymbol{x} \le b, \\ & l_j \le x_j \le u_j, && \forall j \in \{1, 2, \dots, N\}, \\ & f(\boldsymbol{x}) \le f(\bar{\boldsymbol{x}}) - \theta, \\ & x_j \in \mathbb{Z}, && \forall j \in \mathscr{D}. \end{aligned}$$

Proximity search solves it, replaces the incumbent with its solution, and repeats, until the proximity sub-MIP is infeasible.

Algorithm

Algorithm: Proximity search

Input: MIP (10.1) with binary integer variables; incumbent $\bar{\boldsymbol{x}}$; improvement step θ
Output: A solution at least as good as $\bar{\boldsymbol{x}}$

Function *ProximitySearch(MIP, $\bar{\boldsymbol{x}}$, θ)*
  while *true* **do**
    solve $\min\{\Delta(\boldsymbol{x}, \bar{\boldsymbol{x}}) : \boldsymbol{x}$ feasible for the MIP, $f(\boldsymbol{x}) \leq f(\bar{\boldsymbol{x}}) - \theta\}$
    if *the proximity sub-MIP is infeasible* **then**
      return $\bar{\boldsymbol{x}}$ `// no solution better than` $f(\bar{\boldsymbol{x}}) - \theta$
    end
    $\boldsymbol{x}' :=$ its optimal solution
    fix $x_j := x'_j$ for $j \in \mathscr{D}$ and re-optimize f on the continuous variables
    $\bar{\boldsymbol{x}} \leftarrow \boldsymbol{x}'$ `// the closest improving solution`
  end

- **Why minimize the distance.** The original objective is often what makes a MIP hard: its LP relaxation drives the solver toward fractional points far from feasible solutions. In the proximity sub-MIP the objective is the distance, and the LP solution stays close to $\bar{\boldsymbol{x}}$, which is almost feasible: the heuristics of the solver work well, and a feasible solution is usually found quickly.

- In practice the proximity sub-MIP is not solved to optimality: any feasible solution is an improvement.

- **No radius to choose.** The size of the move is decided by the sub-MIP itself: the smallest change that improves the objective by at least θ.

- **Re-optimizing the continuous part.** The solution of the sub-MIP has the right integer part, but its continuous variables were chosen to minimize the distance, not f. Fixing the integer variables and solving the LP on the continuous ones gives the true value of the new incumbent.

- **The step θ.** A large θ asks for large improvements, hence large moves and hard sub-MIPs; a small θ gives easy sub-MIPs and many iterations. With $\theta = 1$ and integer objective values, the method also proves optimality.

Proposition 10.4

If f takes integer values on all the feasible solutions of the MIP and the proximity sub-MIP around $\bar{\boldsymbol{x}}$ with $\theta = 1$ is infeasible, then $f(\bar{\boldsymbol{x}}) = z(I)$.

Proof. The feasible region of the proximity sub-MIP is the set of the feasible solutions $\boldsymbol{x}$ of the MIP with $f(\boldsymbol{x}) \leq f(\bar{\boldsymbol{x}}) - 1$; it does not depend on the objective Δ. If it is empty, every feasible solution has $f(\boldsymbol{x}) > f(\bar{\boldsymbol{x}}) - 1$, hence $f(\boldsymbol{x}) \geq f(\bar{\boldsymbol{x}})$ since both values are integer. So $\bar{\boldsymbol{x}}$ is optimal. □

Example 10.13: Proximity search on the KP

From $\bar{\boldsymbol{x}} = (1, 0, 1, 0)$, of profit 16: the closest solution of profit at least 17 is $(1, 1, 0, 0)$, at distance 2 (item 3 out, item 2 in), of profit 17. Around it, no solution has profit at least 18: by Proposition 10.4, $\{1, 2\}$ is **optimal**.

Example 10.14: Proximity search on $P\,||\,C_{\max}$

From the solution of LPT, of makespan 11:

iteration	cutoff	$\min \Delta$	solution returned	$C_{\max}$
1	$C_{\max} \leq 10$	4	$J_1 = \{5, 6, 7\}$, $J_2 = \{1, 2\}$, $J_3 = \{3, 4\}$	10
2	$C_{\max} \leq 9$	4	$J_1 = \{5, 6, 7\}$, $J_2 = \{1, 4\}$, $J_3 = \{2, 3\}$	9
3	$C_{\max} \leq 8$	—	infeasible	—

By Observation 10.3, distance 4 means two jobs reassigned: jobs 1 and 6 at the first iteration, jobs 2 and 4 at the second. No solution at distance 2 improves the makespan, as for local branching with $k = 2$. At the third iteration the sub-MIP is infeasible, and Proposition 10.4 proves that makespan 9 is optimal.

- The proof of optimality comes from the solver, which proves the infeasibility of the last sub-MIP with its own dual bounds: proximity search, used with $\theta = 1$, is an exact method that improves the incumbent along the way.

10.7 Relax-and-fix and fix-and-optimize

- Many MIPs of production planning, scheduling and logistics are **time-indexed**: their variables are indexed by the periods $t = 1, 2, \ldots, T$ of a planning horizon, and the decisions of a period mainly interact with those of the nearby periods. The two methods of this section exploit this structure: they decompose the MIP along the time, and solve a sequence of small MIPs instead of a large one.

- They are illustrated on the **lot sizing** problem of the MIP modelling course: in each period t a demand d_t must be satisfied, from the production q_t of the period or from the stock s_{t-1} of the previous one; producing in period t requires a set-up, $y_t = 1$, which costs F, and each unit in stock at the end of a period costs h. The production of a period is at most Q (the capacity), or unbounded.

With $M_t = \min\{Q,\ \sum_{\tau=t}^{T} d_\tau\}$ (only the sum for the uncapacitated problem) and $s_0 = 0$:

$$\min \quad \sum_{t=1}^{T} \left(F\, y_t + h\, s_t\right) \tag{10.6a}$$

$$\text{subject to} \quad s_{t-1} + q_t - s_t = d_t, \qquad \forall t \in \{1, 2, \ldots, T\}, \tag{10.6b}$$

$$q_t \le M_t\, y_t, \qquad \forall t \in \{1, 2, \ldots, T\}, \tag{10.6c}$$

$$q_t \ge 0, \qquad \forall t \in \{1, 2, \ldots, T\}, \tag{10.6d}$$

$$s_t \ge 0, \qquad \forall t \in \{1, 2, \ldots, T\}, \tag{10.6e}$$

$$y_t \in \{0, 1\}, \qquad \forall t \in \{1, 2, \ldots, T\}. \tag{10.6f}$$

- Once the set-ups y are fixed, the rest is an LP: the hard decisions are the T binary variables. The LP relaxation is weak, because a fraction $y_t = q_t/M_t$ of a set-up is enough to produce q_t.

10.7.1 Relax-and-fix

Relax-and-fix splits the integer variables into blocks $B_1, B_2, \ldots, B_p$, ordered in time. At step r it solves the MIP in which the variables of B_r are integer, those of $B_1, \ldots, B_{r-1}$ are fixed to the values decided at the previous steps, and those of $B_{r+1}, \ldots, B_p$ are *relaxed* (continuous). Then it fixes the variables of B_r to their values, and moves to the next block. The solution of the last step is feasible.

Algorithm

Algorithm: Relax-and-fix

Input: MIP (10.1); partition $B_1, B_2, \ldots, B_p$ of $\mathscr{D}$, ordered in time
Output: Feasible solution, or FAIL

Function *RelaxAndFix(MIP,* $B_1, \ldots, B_p$*)*
  for $r := 1$ **to** p **do**
    solve the MIP with: x_j fixed for $j \in B_1 \cup \cdots \cup B_{r-1}$, x_j integer for $j \in B_r$, x_j continuous for $j \in B_{r+1} \cup \cdots \cup B_p$
    if *it is infeasible* **then**
      return *FAIL*
    end
    $\boldsymbol{x}' :=$ its optimal solution
    fix $x_j := x'_j$ for all $j \in B_r$
  end
  return $\boldsymbol{x}'$ `// all the integer variables are integer`

- **Why.** The decisions of the first periods are taken with care (integer), while the future is only estimated (relaxed). Each MIP has the integer variables of a single block, so it is small.

- **Myopia.** The relaxed future is optimistic: a fraction of a set-up costs a fraction of F. A decision that looks good because the future is cheap may turn out to be bad when the future becomes integer. With capacities, the fixed decisions can even make a later step infeasible.

- **Variants.** Larger blocks give better solutions and harder MIPs; *overlapping* blocks solve $B_r \cup B_{r+1}$ as integer but fix only B_r, to see a little further into the future.

Example 10.15: Relax-and-fix on the lot sizing instance of the MIP course

The instance of the MIP course has $T = 5$, $d = (20, 10, 30, 40, 10)$, $F = 50$, $h = 1$, no capacity. Its LP relaxation has value $\frac{36985}{396} \approx 93.40$, the constructive heuristic least unit cost of the course gives 200, and the optimal value is 170. Relax-and-fix with one period per block, $B_t = \{y_t\}$, finds a solution of value 170: **optimal**.

Example 10.16: Relax-and-fix on a capacitated instance

Consider $T = 6$, $d = (50, 30, 10, 10, 40, 10)$, $F = 40$, $h = 1$ and capacity $Q = 80$. The LP relaxation has value $\frac{2150}{21} \approx 102.38$ and the optimal value is 160, with set-ups in periods 1, 2, 5. Relax-and-fix with $B_t = \{y_t\}$:

step r	value of the MIP of step r	y of the solution	fixing
1	$\frac{2465}{21} \approx 117.38$	$(1,\ 0.375,\ 0.143,\ 0.167,\ 1,\ 0)$	$y_1 := 1$
2	$\frac{2780}{21} \approx 132.38$	$(1,\ 0,\ 0.143,\ 0.167,\ 1,\ 0)$	$y_2 := 0$
3	$\frac{500}{3} \approx 166.67$	$(1,\ 0,\ 1,\ 0.167,\ 1,\ 0)$	$y_3 := 1$
4	170	$(1, 0, 1, 0, 1, 0)$	$y_4 := 0$
5	170	$(1, 0, 1, 0, 1, 0)$	$y_5 := 1$
6	170	$(1, 0, 1, 0, 1, 0)$	$y_6 := 0$

(the fractional values are rounded to three decimals). The result has set-ups in periods 1, 3, 5: production 80, 20, 50, stocks 30, 10, 10 at the end of periods 1, 3, 5, cost $3 \cdot 40 + 50 = 170$. The wrong decision is taken at step 2: with the future relaxed, $y_2 = 0$ gives 132.38 and $y_2 = 1$ gives 142.38, so relax-and-fix saves the set-up of period 2; but then the capacity forces a set-up in period 3, which the relaxation had paid only for a fraction 0.143.

10.7.2 Fix-and-optimize

Fix-and-optimize improves an incumbent $\bar{\boldsymbol{x}}$. It considers a sequence of *windows* $W_1, W_2, \ldots, W_q \subseteq \mathscr{D}$, typically consecutive periods. For each window it fixes all the integer variables *outside* the window to their values in $\bar{\boldsymbol{x}}$, and solves the sub-MIP on the variables of the window, with the cutoff constraint. A better solution becomes the incumbent. The passes over the windows are repeated until a whole pass gives no improvement.

Algorithm

Algorithm: Fix-and-optimize

Input: MIP (10.1); incumbent $\bar{\boldsymbol{x}}$; windows $W_1, \ldots, W_q$; improvement step θ
Output: A solution at least as good as $\bar{\boldsymbol{x}}$

Function *FixAndOptimize(MIP,* $\bar{\boldsymbol{x}}$*,* $W_1, \ldots, W_q$*,* θ*)*
  improved := true
  while *improved* **do**
    improved ← false
    for $r := 1$ **to** q **do**
      solve the MIP with $x_j := \bar{x}_j$ for $j \in \mathscr{D} \setminus W_r$ and $f(\boldsymbol{x}) \le f(\bar{\boldsymbol{x}}) - \theta$
      if *a feasible solution* $\boldsymbol{x}'$ *is found* **then**
        $\bar{\boldsymbol{x}} \leftarrow \boldsymbol{x}'$; *improved* ← true
      end
    end
  end
  return $\bar{\boldsymbol{x}}$

- Fix-and-optimize is a **local search** whose moves change all the integer variables of a window at once, explored exactly by the solver: the integer-programming counterpart of the neighbourhoods of the notes on local search. It stops in a local optimum with respect to all the windows.

- Relax-and-fix and fix-and-optimize are natural partners: the first builds a solution looking forward in time, the second repairs the myopic decisions looking at the past and the future together.

Example 10.17: Fix-and-optimize on the capacitated instance

Start from the solution of relax-and-fix of Example 10.16, $\bar{\boldsymbol{y}} = (1, 0, 1, 0, 1, 0)$ of cost 170, with the windows of two consecutive periods $W_t = \{y_t, y_{t+1}\}$, $t = 1, 2, \ldots, 5$.

window	incumbent after the window	
$\{y_1, y_2\}$	170	no improvement
$\{y_2, y_3\}$	160	$\bar{\boldsymbol{y}} = (1, 1, 0, 0, 1, 0)$
$\{y_3, y_4\}$, $\{y_4, y_5\}$, $\{y_5, y_6\}$	160	no improvement
second pass, all windows	160	no improvement: stop

The window $\{y_2, y_3\}$ revises the wrong decision of relax-and-fix: the set-up moves from period 3 to period 2. The plan produces 50, 50, 50 in periods 1, 2, 5, with stocks 20 and 10 at the end of periods 2, 3 and 10 at the end of period 5: cost $3 \cdot 40 + 40 = 160$, the **optimum**.

10.8 Heuristics inside the solver

- A modern MIP solver is not only a branch-and-bound algorithm: it runs, automatically, most of the heuristics of these notes. In Gurobi (the chapter on Gurobi of the MIP modelling course explains how to use it from Python):
  - *before the root LP*, heuristics that need no relaxation look for a first solution;
  - *at the root*, after the LP, rounding, several diving heuristics and the feasibility pump are run;
  - *during the branch-and-bound*, RINS and other sub-MIP heuristics are run at some nodes, with the LP solution of the node, and the new incumbents are reported in the log by a line starting with `H` (heuristic) instead of `*` (integer LP solution of a node).

- The effort spent in heuristics is controlled by parameters. The most important ones are:

parameter	default	meaning
`Heuristics`	0.05	fraction of the work spent in the heuristics (0 to 1)
`MIPFocus`	0	1: focus on finding good feasible solutions rather than on the bound
`RINS`	-1	frequency of RINS (every n nodes; -1 automatic, 0 off)
`PumpPasses`	-1	passes of the feasibility pump at the root (-1 automatic)
`SubMIPNodes`	500	node limit of the sub-MIPs of RINS and similar heuristics
`NoRelHeurWork`	0	work spent in the heuristic without relaxation before the root
`ImproveStartGap`	0	when the gap reaches this value, switch to improving the incumbent only

- A heuristic solution computed outside the solver — by FFD, LPT, a local search — can be passed to it as a **MIP start** (the attribute `Start` of the variables): the solver starts from it as incumbent, uses it for pruning and for RINS, and never returns anything worse.

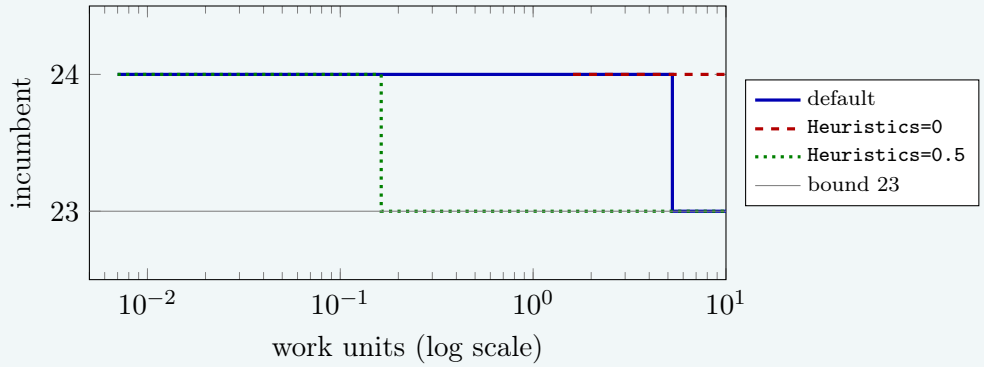
Example 10.18: The effect of the heuristics of Gurobi on a medium BPP instance

The instance has $n = 60$ items, with weights $w_j = 20 + (s_j \bmod 41)$, where $s_1, s_2, \ldots$ is the Park–Miller sequence from the seed 1, and capacity $c = 100$:

58	29	22	22	55	45	38	40	34	32
43	57	20	57	53	32	40	57	50	33
29	49	37	20	27	22	55	40	48	40
52	24	21	48	31	36	38	46	30	20
31	58	40	28	54	50	26	31	27	54
26	26	22	30	33	55	27	50	47	26

The total weight is 2271, so $LB = \lceil 2271/100 \rceil = 23$; FFD uses 24 bins, and the model is built with $|\mathscr{B}| = 24$ bins. Gurobi 13.0.3 is run on one thread with a limit of 10 *work units* (a deterministic measure of the computing effort, roughly a second each), with different parameters.

parameters	incumbent	bound	nodes	work	incumbents found (work: value)
default	23	23	2533	5.27	0.007: 24; 5.273: 23
`Heuristics=0`	24	23	4809	10.00	1.610: 24
`Heuristics=0.5`	23	23	1	0.16	0.007: 24; 0.163: 23
`MIPFocus=1`	23	23	1	0.16	0.007: 24; 0.160: 23
`NoRelHeurWork=1`	23	23	2533	6.48	0.007: 24; 6.477: 23
`RINS=0`	23	23	2959	5.86	0.007: 24; 5.861: 23



With the default parameters, a solution of 24 bins is found at once, and the optimal one only after 2533 nodes. Spending more effort in the heuristics (`Heuristics=0.5` or `MIPFocus=1`) finds 23 at the root, about 30 times faster; without heuristics the first solution arrives after 1.61 work units, and the optimum is not found within the limit. Since the bound is 23 from the root, the whole difficulty of this instance is primal: finding the solution, not proving it optimal.

- The numbers depend on the instance, on the version of the solver and on the machine; a single instance proves nothing (notes on definitions). The lesson is qualitative: on instances whose bound is strong, as the BPP with $LB = z(I)$, the time to solve is the time to find a good solution, and the heuristics of the solver — or a good MIP start — decide it.

10.9 What MIP-based heuristics give

- The notes on definitions ended with a warning: a heuristic gives only one side of the gap, and without a dual bound nothing can be said about the quality of its solution. The heuristics of the previous notes give *only* primal bounds.

- A MIP-based heuristic works with a MIP model, and the model comes with its LP relaxation; run inside a MIP solver, it comes with the dual bound of the branch-and-bound as well. For the first time, the heuristic **knows how far it is from the optimum** while it runs: the gap of the notes on definitions is computed at every moment, and when it is 0 the solution is *proved* optimal.

	heuristics of the previous notes			MIP-based heuristics		
instance	solution	dual bound	gap	solution	dual bound	gap
KP	GR: 16	LP: 17	5.9%	LB, PS: 17	PS: 17	0%
BPP	FF: 4	$\lceil 21/7 \rceil = 3$	33.3%	pump (seed 7): 3	LP: 3	0%
TSP	NN: 25	two-edge: 17	47.1%	rounding: 18	LP: 18	0%
$P \,\|\|\, C_{\max}$	LPT: 11	9	22.2%	RINS, LB, PS: 9	LP: 9	0%
lot sizing, capacitated	RF: 170	LP: 102.38	66.0%	FO: 160	solver: 160	0%
medium BPP	FFD: 24	23	4.3%	Gurobi: 23	23	0%

- On the KP the integer rounding of the LP value is $\lfloor 17.75 \rfloor = 17$, and proximity search proves the optimality of 17 by itself (Proposition 10.4). On the capacitated lot sizing instance the LP bound is weak, and the gap is closed by the branch-and-bound of the solver, which proves that 160 is optimal.

A MIP-based heuristic does not only find solutions: inside a MIP solver it works with a dual bound, so the gap of its solution is always known, and a gap equal to 0 certifies optimality.

- The methods of these notes, in summary:

method	needs LP	needs incumbent	fixing	idea
rounding	yes	no	—	round the LP solution
diving	yes	guided: yes	one at a time	fix a variable, re-solve the LP, repeat
feasibility pump	yes	no	—	alternate rounding and projection until they meet
RINS	yes	yes	hard	fix where LP and incumbent agree, solve the sub-MIP
local branching	no	yes	soft	sub-MIP with $\Delta(\boldsymbol{x}, \bar{\boldsymbol{x}}) \leq k$
proximity search	no	yes	soft	minimize $\Delta(\boldsymbol{x}, \bar{\boldsymbol{x}})$ subject to the cutoff
relax-and-fix	yes	no	by blocks	integer now, relaxed later, then fix
fix-and-optimize	no	yes	by windows	re-optimize a window, the rest fixed

10.10 Python implementation

- The codes use Python 3 and `gurobipy`. The variables of a model are accessed by their position j in `m.getVars()`, and the list `I` of the positions of the integer variables is the set $\mathscr{D}$; indices start from 0.

- The module `common.py` contains the environment of the solver and the operations used by all the methods: the LP relaxation, the rounding $[\cdot]$, the completion of a point, the distance Δ and the cutoff constraint.

```
import math
import gurobipy as gp
from gurobipy import GRB

EPS = 1e-6                                  # integrality and comparison tolerance

ENV = gp.Env(empty=True)                    # one silent, single-thread (deterministic) environment
ENV.setParam("OutputFlag", 0)
ENV.setParam("Threads", 1)
ENV.start()


def int_index(m):
    """positions j (in m.getVars()) of the integer and binary variables: the set I"""
    return [j for j, v in enumerate(m.getVars()) if v.VType != GRB.CONTINUOUS]


def nearest(v):
    """[v] = floor(v + 1/2): rounding to the nearest integer, 1/2 rounded up"""
    return math.floor(v + 0.5 + EPS)


def is_integral(v):
    return abs(v - round(v)) <= EPS


def solve_lp(m):
    """optimal solution of the LP relaxation of m (with its current bounds), None if infeasible"""
    m.update()
    r = m.relax()
    r.optimize()
    if r.Status != GRB.OPTIMAL:
        return None
    return [v.X for v in r.getVars()]
```

```
35
36
37 def obj_of(m, x):
38     """value of the objective function of m at the point x"""
39     e = m.getObjective()
40     return e.getConstant() + sum(e.getCoeff(k) * x[e.getVar(k).index]
           for k in range(e.size()))
41
42
43 def round_point(x_star, I):
44     """x~ = [x*]: round the integer components, copy the continuous
           ones"""
45     x_tilde = list(x_star)
46     for j in I:
47         x_tilde[j] = nearest(x_star[j])
48     return x_tilde
49
50
51 def evaluate(m, x_int):
52     """fix the integer variables to x_int and optimize the continuous
           ones:
53         returns (value, x), or None if the point cannot be completed to
               a feasible solution"""
54     f = m.copy()
55     V = f.getVars()
56     for j in int_index(f):
57         V[j].LB = V[j].UB = x_int[j]
58     f.optimize()
59     if f.Status != GRB.OPTIMAL:
60         return None
61     return f.ObjVal, [v.X for v in V]
62
63
64 def solve_mip(m):
65     """optimal value and solution of the MIP m, None if infeasible"""
66     m.optimize()
67     if m.SolCount == 0:
68         return None
69     return m.ObjVal, [v.X for v in m.getVars()]
70
71
72 def distance(m, x_tilde, I):
73     """the linear expression Delta(x, x~) = sum_{j in I} |x_j - x~_j|
           in the variables of m:
74         x_j - l_j if x~_j is at its lower bound, u_j - x_j if at its
               upper bound,
75         otherwise an auxiliary variable d_j >= |x_j - x~_j|"""
76     V = m.getVars()
77     expr = gp.LinExpr()
78     for j in I:
79         if x_tilde[j] == V[j].LB:
80             expr += V[j] - x_tilde[j]
81         elif x_tilde[j] == V[j].UB:
```

```
            expr += x_tilde[j] - V[j]
        else:
            d = m.addVar(name=f"d[{j}]")
            m.addConstr(d >= V[j] - x_tilde[j])
            m.addConstr(d >= x_tilde[j] - V[j])
            expr += d
    return expr


def delta(x, x_tilde, I):
    """value of Delta(x, x~)"""
    return sum(abs(x[j] - x_tilde[j]) for j in I)


def add_cutoff(m, z, theta):
    """require a solution better than z by at least theta"""
    obj = m.getObjective()
    if m.ModelSense == GRB.MINIMIZE:
        m.addConstr(obj <= z - theta, name="cutoff")
    else:
        m.addConstr(obj >= z + theta, name="cutoff")


def note(trace, *items):
    """record a step of the algorithm (used to print the examples of
       the notes)"""
    if trace is not None:
        trace.append(items)
```

- The models of Section 10.2 and of the lot sizing problem (`models.py`):

```
from itertools import combinations


def build_kp(p, w, c):
    m = gp.Model("KP", env=ENV)
    n = len(p)
    x = m.addVars(n, vtype=GRB.BINARY, name="x")
    m.setObjective(gp.quicksum(p[i] * x[i] for i in range(n)), GRB.
       MAXIMIZE)
    m.addConstr(gp.quicksum(w[i] * x[i] for i in range(n)) <= c, name="
       capacity")
    m.update()
    return m


def build_bpp(w, c, nb):
    m = gp.Model("BPP", env=ENV)
    n = len(w)
    x = m.addVars(n, nb, vtype=GRB.BINARY, name="x")
    y = m.addVars(nb, vtype=GRB.BINARY, name="y")
    m.setObjective(y.sum(), GRB.MINIMIZE)
```

```
20      m.addConstrs((x.sum(i, "*") == 1 for i in range(n)), name="assign")
21      m.addConstrs((gp.quicksum(w[i] * x[i, b] for i in range(n)) <= c *
            y[b]
22                    for b in range(nb)), name="capacity")
23      m.update()
24      return m
25
26
27  def build_tsp(d):
28      """symmetric TSP: one binary variable per edge, degree and subtour
            elimination constraints"""
29      m = gp.Model("TSP", env=ENV)
30      n = len(d)
31      E = [(i, j) for i in range(n) for j in range(i + 1, n)]
32      x = m.addVars(E, vtype=GRB.BINARY, name="x")
33      m.setObjective(gp.quicksum(d[i][j] * x[i, j] for i, j in E), GRB.
            MINIMIZE)
34      m.addConstrs((gp.quicksum(x[e] for e in E if i in e) == 2 for i in
            range(n)), name="degree")
35      for k in range(2, n - 1):
36          for S in combinations(range(n), k):
37              m.addConstr(gp.quicksum(x[i, j] for i, j in E if i in S and
                    j in S) <= len(S) - 1)
38      m.update()
39      return m
40
41
42  def build_pcm(t, m):
43      model = gp.Model("PCmax", env=ENV)
44      n = len(t)
45      x = model.addVars(n, m, vtype=GRB.BINARY, name="x")
46      Cmax = model.addVar(name="Cmax")
47      model.setObjective(Cmax, GRB.MINIMIZE)
48      model.addConstrs((x.sum(j, "*") == 1 for j in range(n)), name="
            assign")
49      model.addConstrs((gp.quicksum(t[j] * x[j, h] for j in range(n)) <=
            Cmax
50                        for h in range(m)), name="load")
51      model.update()
52      return model
53
54
55  def build_lot_sizing(d, f, h, cap=None):
56      """single-item lot sizing: production q_t, stock s_t, set-up y_t;
            capacity cap (None = uncapacitated)"""
57      m = gp.Model("LS", env=ENV)
58      T = len(d)
59      y = m.addVars(T, vtype=GRB.BINARY, name="y")
60      q = m.addVars(T, name="q")
61      s = m.addVars(T, name="s")
62      m.setObjective(f * y.sum() + h * s.sum(), GRB.MINIMIZE)
63      for t in range(T):
64          m.addConstr((s[t - 1] if t > 0 else 0) + q[t] - s[t] == d[t],
```

```
            name=f"balance[{t}]")
        M = sum(d[t:]) if cap is None else min(cap, sum(d[t:]))
        m.addConstr(q[t] <= M * y[t], name=f"setup[{t}]")
    m.update()
    return m


def build_ip2(A, b, c, U):
    """max c1 x1 + c2 x2 subject to A x <= b, 0 <= x <= U, x integer"""
    m = gp.Model("IP2", env=ENV)
    x = m.addVars(2, lb=0, ub=U, vtype=GRB.INTEGER, name="x")
    m.setObjective(c[0] * x[0] + c[1] * x[1], GRB.MAXIMIZE)
    for k in range(len(A)):
        m.addConstr(A[k][0] * x[0] + A[k][1] * x[1] <= b[k])
    m.update()
    return m


def pcm_solution(m, sets):
    """the solution of the P||Cmax model with job sets J_1, ..., J_m
       (1-based jobs)"""
    x = [0] * m.NumVars
    index = {v.VarName: k for k, v in enumerate(m.getVars())}
    for h, J in enumerate(sets):
        for j in J:
            x[index[f"x[{j - 1},{h}]"]] = 1
    return evaluate(m, x)[1]
```

- Rounding and diving:

```
def rounding(m, trace=None):
    I = int_index(m)
    x_star = solve_lp(m)                          # optimal solution of
       the LP relaxation
    note(trace, "LP", x_star)
    x_tilde = round_point(x_star, I)              # x~_j = [x*_j] for
       every integer variable
    note(trace, "rounded", x_tilde)
    return evaluate(m, x_tilde)                   # (value, x) if feasible
       , None otherwise
```

```
def diving(m, rule="fractional", x_inc=None, trace=None):
    d = m.copy()                                  # the bounds of the copy
        are changed, not those of m
    V = d.getVars()
    I = int_index(d)
    while True:
        x_star = solve_lp(d)                      # LP relaxation with the
            current bounds
        if x_star is None:
            note(trace, "LP infeasible")
```

```
            return None                              # the dive fails
        F = [j for j in I if not is_integral(x_star[j])]
        note(trace, "LP", obj_of(d, x_star), x_star)
        if not F:
            return evaluate(m, round_point(x_star, I))
        if rule == "fractional":                     # the least fractional
            variable, to its nearest integer
            j = min(F, key=lambda k: round(abs(x_star[k] - nearest(
                x_star[k])), 9))
            v = nearest(x_star[j])
        else:                                        # guided: the variable
            closest to the incumbent, to its value
            j = min(F, key=lambda k: round(abs(x_star[k] - x_inc[k]),
                9))
            v = x_inc[j]
        note(trace, "fix", V[j].VarName, v)
        V[j].LB = V[j].UB = v                        # fix x_j := v
```

- The feasibility pump:

```
def projection(m, x_tilde, I):
    """x* = argmin { Delta(x, x~) : x in P }, P the LP relaxation of m"
        ""
    p = m.relax()
    p.setObjective(distance(p, x_tilde, I), GRB.MINIMIZE)
    p.optimize()
    return [v.X for v in p.getVars()][:m.NumVars]

def feasibility_pump(m, max_iter=100, seed=1, trace=None):
    I = int_index(m)
    x_star = solve_lp(m)                             # x* = optimal solution
        of the LP relaxation
    note(trace, "LP", x_star)
    if all(is_integral(x_star[j]) for j in I):
        return evaluate(m, x_star)
    x_tilde = round_point(x_star, I)                 # x~ = [x*]
    note(trace, "round", x_tilde)
    history = [x_tilde]                              # the points x~ visited
        so far
    for it in range(1, max_iter + 1):
        x_star = projection(m, x_tilde, I)           # the point of P closest
            to x~
        dist = delta(x_star, x_tilde, I)
        note(trace, "project", x_star, dist)
        if dist <= EPS:                              # x* = x~ is integer and
            feasible
            return evaluate(m, x_tilde)
        x_new = round_point(x_star, I)
        if all(x_new[j] == x_tilde[j] for j in I):           # short
            cycle: [x*] = x~
            j = max(I, key=lambda k: (round(abs(x_star[k] - x_tilde[k])
                , 9), -k))
```

```
            x_new[j] += 1 if x_star[j] > x_tilde[j] else -1  # flip the most uncertain component
            note(trace, "flip", x_new, j)
        else:
            note(trace, "round", x_new)
        if any(all(x_new[j] == y[j] for j in I) for y in history[-3:]):  # long cycle
            for j in I:                                  # restart: random flips (binary variables)
                seed = (16807 * seed) % 2147483647
                rho = seed / 2147483647 - 0.3  # rho_j uniform in [-0.3, 0.7]
                if abs(x_star[j] - x_new[j]) + max(rho, 0) > 0.5:
                    x_new[j] = 1 - x_new[j]
            note(trace, "restart", x_new)
        x_tilde = x_new
        history.append(x_tilde)
    return None                                          # no feasible solution found
```

- RINS, local branching and proximity search:

```
def rins(m, x_tilde, theta=1, trace=None):
    I = int_index(m)
    z_tilde = obj_of(m, x_tilde)                         # value of the incumbent
    x_star = solve_lp(m)                                 # optimal solution of the LP relaxation
    s = m.copy()
    V = s.getVars()
    fixed = []
    for j in I:
        if abs(x_star[j] - x_tilde[j]) <= EPS:           # LP and incumbent agree on x_j
            V[j].LB = V[j].UB = x_tilde[j]
            fixed.append(j)
    note(trace, "LP", x_star, "fixed", [V[j].VarName for j in fixed])
    add_cutoff(s, z_tilde, theta)                        # look only for better solutions
    res = solve_mip(s)                                   # the sub-MIP on the free variables
    if res is None:
        note(trace, "sub-MIP infeasible")
        return z_tilde, x_tilde
    note(trace, "sub-MIP", res[0])
    return res
```

```
def local_branching(m, x_tilde, k, k_max, k_step, theta=1, trace=None):
    I = int_index(m)
    w = m.copy()                                         # the master MIP, which receives the reverse constraints
    z_tilde = obj_of(m, x_tilde)
```

```
    while k <= k_max:
        s = w.copy()
        s.addConstr(distance(s, x_tilde, I) <= k, name="local_branch")
              # Delta(x, x~) <= k
        add_cutoff(s, z_tilde, theta)
        res = solve_mip(s)
        w.addConstr(distance(w, x_tilde, I) >= k + 1)
                                # never explore it again
        if res is not None:                         # better solution: re-
            center the neighbourhood
            note(trace, "improved", k, res[0], res[1])
            z_tilde, x_tilde = res
        else:                                       # no better solution
            within distance k: enlarge it
            note(trace, "no improvement", k)
            k += k_step
    return z_tilde, x_tilde
```

```
def proximity_search(m, x_tilde, theta=1, trace=None):
    I = int_index(m)
    z_tilde = obj_of(m, x_tilde)
    while True:
        p = m.copy()
        add_cutoff(p, z_tilde, theta)           # better than the
            incumbent by at least theta
        p.setObjective(distance(p, x_tilde, I), GRB.MINIMIZE)   # as
            close as possible to it
        res = solve_mip(p)
        if res is None:                         # no solution better
            than z~ - theta exists
            note(trace, "infeasible")
            return z_tilde, x_tilde
        z_new, x_new = evaluate(m, res[1])      # value of the new
            solution, continuous part re-optimized
        note(trace, "found", res[0], z_new, x_new)
        z_tilde, x_tilde = z_new, x_new
```

- Relax-and-fix and fix-and-optimize:

```
def relax_and_fix(m, blocks, trace=None):
    r = m.copy()
    V = {v.VarName: v for v in r.getVars()}
    vtype = {name: V[name].VType for B in blocks for name in B}
    for B in blocks:                            # at first, all the
        blocks are relaxed
        for name in B:
            V[name].VType = GRB.CONTINUOUS
    for B in blocks:
        for name in B:                          # the current block is
            integer again
            V[name].VType = vtype[name]
```

```
        res = solve_mip(r)
        if res is None:
            note(trace, "infeasible")
            return None
        note(trace, "block", B, res[0], {n: V[n].X for BB in blocks for
            n in BB})
        for name in B:                          # fix the current block
           to its value
            V[name].LB = V[name].UB = round(V[name].X)
    return res
```

```
def fix_and_optimize(m, x_tilde, windows, theta=1, trace=None):
    I = int_index(m)
    names = [v.VarName for v in m.getVars()]
    z_tilde = obj_of(m, x_tilde)
    improved = True
    while improved:                             # one pass over all the
       windows
        improved = False
        for W in windows:
            s = m.copy()
            V = s.getVars()
            for j in I:
                if names[j] not in W:           # outside the window:
                   fixed to the incumbent
                    V[j].LB = V[j].UB = x_tilde[j]
            add_cutoff(s, z_tilde, theta)
            res = solve_mip(s)
            if res is not None:
                z_tilde, x_tilde = res
                improved = True
            note(trace, "window", W, z_tilde)
    return z_tilde, x_tilde
```

- The experiment of Example 10.18, with a callback that records every new incumbent:

```
def run_with_parameters(m, params, work_limit):
    """solve m with the given Gurobi parameters for at most work_limit
       work units;
       returns incumbent, dual bound, nodes, work and the list of the
          incumbents found (work, value)"""
    s = m.copy()
    s.Params.WorkLimit = work_limit
    for name, value in params.items():
        s.setParam(name, value)
    found = []
    def callback(model, where):                 # called by Gurobi at
       every new incumbent
        if where == GRB.Callback.MIPSOL:
            found.append((model.cbGet(GRB.Callback.WORK), model.cbGet(
               GRB.Callback.MIPSOL_OBJ)))
```

```
    s.optimize(callback)
    z = s.ObjVal if s.SolCount > 0 else None
    return z, s.ObjBound, int(s.NodeCount), s.Work, found
```

- All the functions of this section are in the folder `MIP_HEUR_CODE`, together with `instances.py`, which contains the instances of the notes and of the exercises. The command `python3 run_tests.py` runs every example of these notes and every instance of the exercises, checking each value against the expected value, written in the file.

10.11 Typical mistakes

Warning

- **Rounding a problem with equality constraints.** Independent roundings of an assignment constraint almost never sum to 1: on the BPP and on $P\,||\,C_{\max}$ rounding fails.
- **Believing that a dive can undo a decision.** A dive is one path of the tree, without backtracking: the first wrong fixing (on the KP, $x_2 := 0$) is never revised.
- **Using the feasibility pump to get a good solution.** After the first LP the pump ignores the objective: it returns a feasible solution, possibly with empty bins left open.
- **Trusting hard fixing.** RINS fixes the variables on which the LP and the incumbent agree, so it cannot find a better solution that changes one of them.
- **Forgetting the cutoff constraint.** Without $f(\boldsymbol{x}) \leq f(\bar{\boldsymbol{x}}) - \theta$ the sub-MIP returns the incumbent solution again; with it, an infeasible sub-MIP proves a local optimum.
- **Claiming optimality without the conditions.** Proposition 10.4 needs $\theta = 1$ and integer objective values on all the feasible solutions.
- **Ignoring the myopia of relax-and-fix.** The relaxed future is optimistic: a decision that looks cheap can force a set-up later.
- **Generalizing from one instance or one solver run.** Paths and times depend on the version of the solver and on the tie-breaking rules, so the lessons are qualitative.

10.12 In summary

In summary

- MIP-based heuristics use the LP relaxation to decide where to look and a MIP solver to explore a large neighbourhood exactly.
- Rounding and diving build a solution from the LP; the feasibility pump looks only for feasibility; RINS, local branching and proximity search improve an incumbent solution with a sub-MIP.
- Relax-and-fix builds a solution along the time, fix-and-optimize improves it window by window.
- Inside a MIP solver the dual bound is available: the gap of the heuristic solution is known, and a gap equal to 0 certifies optimality.

Dashboard of the reference instances. Best solution found by the methods of this chapter against the dual bound (max for the KP, min for the others):

instance	solution	dual bound	gap
KP	17	17	0%
BPP	3	3	0%
$P \mid\mid C_{\max}$	9	9	0%
lot sizing, capacitated	160	160	0%
medium BPP	23	23	0%

Bibliography

[1] T. Achterberg, T. Berthold, Improving the feasibility pump, *Discrete Optimization* 4 (2007) 77–86.

[2] L. Bertacco, M. Fischetti, A. Lodi, A feasibility pump heuristic for general mixed-integer problems, *Discrete Optimization* 4 (2007) 63–76.

[3] E. Danna, E. Rothberg, C. Le Pape, Exploring relaxation induced neighborhoods to improve MIP solutions, *Mathematical Programming* 102 (2005) 71–90.

[4] M. Fischetti, F. Glover, A. Lodi, The feasibility pump, *Mathematical Programming* 104 (2005) 91–104.

[5] M. Fischetti, A. Lodi, Local branching, *Mathematical Programming* 98 (2003) 23–47.

[6] M. Fischetti, M. Monaci, Proximity search for 0-1 mixed-integer convex programming, *Journal of Heuristics* 20 (2014) 709–731.